Java Brew

Auction Website

Software Design Document

Deliverable 2 Addendum

Revisions

Version	Date	Author(s)	Summary of Changes
1.0	1/26/26	Yazan Haddad, Lama Abdelfattah, Mahmoud Dahlan, Tristan Augustus, Amanullah Asim	Initial Version: Creation of the SDD and adding the table of contents and all major sections for organization.
1.1	1/29/26	Mahmoud Dahlan, Tristan Augustus, Amanullah Asim	Completed the Introduction and Major Design Decisions sections to the document
1.2	1/31/26	Yazan Haddad, Lama Abdelfattah	Completed and added the Test Driven Development and AI Acknowledgement sections
1.3	2/2/26	Amanullah Asim, Lama Abdelfattah	Completed and added the Sequence Diagram and Activities Diagram sections of the document
1.4	2/3/26	Tristan Augustus, Mahmoud Dahlan, Yazan Haddad	Completed and added the Architecture and Activities Plan sections
1.5	2/6/26	Yazan Haddad, Lama Abdelfattah, Mahmoud Dahlan, Tristan Augustus, Amanullah Asim	Final Version for submission: Updated the revision history and the Group Meeting Logs
2.1	2/18/26	Lama Abdelfattah, Tristan Augustus, Amanullah Asim	Appended the sequence diagrams to better represent the implementations use cases
2.2	2/22/26	Tristan Augustus, Yazan Haddad	Appended activities plan section to better represent what was completed during implementation
2.3	2/25/26	Yazan Haddad, Tristan Augustus, Amanullah Asim	Appended the architecture section to represent the services and interfaces used in the implementation. Appended the diagrams associated with the architecture including class diagrams
2.4	3/1/26	Lama Abdelfattah, Tristan Augustus,	Appended the test case summary to explain the postman test and test cases made
2.5	3/3/26	Mahmoud Dahlan, Tristan Augustus	Appended key design choices to explain the 4 design patterns used and 2 unused and appended sections 11 and 12
2.6	3/6/26	Yazan Haddad, Lama Abdelfattah, Mahmoud Dahlan, Tristan Augustus, Amanullah Asim	Final Version for submission updated/appended revision history and meeting log

Table of Contents

Revisions
Table of Contents
1 Introduction
    1.1 Purpose
    1.2 Overview
    1.3 References
2 Major Design Decisions
    Key Design Choices
    (Appended) Key Design Choices
3 Sequence Diagrams
    UC1: Sign Up and Sign In
    UC5: Payment
    UC2.1: Search Auction Items
    UC3: Place a Bid
3 (Appended) Sequence Diagrams
    UC1: Sign-up and Login
    UC2: Browse and Search Catalogue
    UC3: Placing and Validating Bids
    UC4: Auction End Behaviour
    UC5: Payment
    UC6: Receipt and Shipping
    UC7: Creating An Auction Listing
4 Activity Diagrams
5 Architecture
    5.1 Decomposition Diagram
    5.2 Services Table
    5.3 Interfaces Table
5 (Appended) Architecture

5.1 (Appended) Decomposition Diagram
5.2 (Appended) Services Table
5.3 (Appended) Interfaces Table
6 Activities Plan
6.1 Product Backlog for Auction System
6.1 (Appended) Product Backlog for Auction System
6.2 Deliverable 1 Sprint Backlog
6.3 Deliverable 2 Sprint Backlog
6.3 (Appended) Deliverable 2 Sprint Backlog
6.4 Deliverable 3 Sprint Backlog
6.5 Gantt Diagram
7 Group Meeting Logs
7 (Appended) Group Meeting Logs
8 Test Driven Development (TDD)
8.1 User Authentication
8.2 User Registration
8.3 Data Entry Form
8.4 Reporting
8.5 User Profile
9 (Appended) Test plan Summary
10 AI Acknowledgement
11 (Appended) Deviations from Original Design
12 (Appended) Amendments to the Original Document

1 Introduction

1.1 Purpose

The purpose of this project is to design and later implement a web-based auction e-commerce system that supports forward auctions. The system allows users to register, browse auction items, place bids in real time, and complete payments for items they win. This Software Design Document (SDD) defines the high-level design, architecture, and development plan for the system.

1.2 Overview

The auction platform enables sellers to list items for sale using forward auctions, where bidders compete by placing increasingly higher bids until a fixed auction time expires. Users can sign up and sign in, search for auction items, view current bid prices and remaining time, and place bids on selected items. When an auction ends, the highest bidder is declared the winner and proceeds to payment. After payment is completed, the system provides a receipt and shipping details.

The system is designed using modular services to ensure scalability, maintainability, and ease of future deployment as microservices. This deliverable focuses on design and planning only; implementation will be completed in later deliverables.

1.3 References

EECS 4413 Project Description Document – Auction E-Commerce System
UML 2.x Specification

2 Major Design Decisions

Key Design Choices

The system is designed as a multi-tier web application consisting of a front-end user interface, a gateway service, and several back-end services responsible for authentication, catalogue management, auctions, and payments. Separating responsibilities into dedicated services simplifies development and allows independent scaling of system components.

Modularity: High Cohesion and Low Coupling

Each service in the system has a single well-defined responsibility:

- The **IAM Service** handles user registration and authentication.
- The **Catalogue Service** manages auction item listings and search.
- The **Auction Service** handles bidding logic and auction timing.
- The **Payment Service** manages payment processing and receipt generation.

This separation ensures high cohesion, since each module focuses on one functional area, and low coupling, since services interact only through clearly defined interfaces rather than direct internal dependencies.

Chosen Architectural Pattern

A **Layered Service-Oriented Architecture (SOA)** combined with a **Gateway (Facade) Pattern** is chosen.

- The UI layer handles user interaction.
- The Gateway Service acts as a single entry point that routes requests to back-end services.
- The Back-End Services layer contains IAM, Catalogue, Auction, and Payment services.
- The Data layer stores persistent information in databases.

This architecture improves maintainability, supports future microservice deployment, and allows services to evolve independently.

Rejected Architectural Pattern

A **Monolithic Architecture** was considered but rejected. In a monolithic design, all system components would be tightly integrated into a single application. This would make future scaling difficult, increase maintenance complexity, and reduce flexibility in deploying services independently. Therefore, a modular service-based approach was selected instead.

(Appended) Key Design Choices

Design Patterns Used

The implementation of the JavaBrew auction backend applies several design patterns to improve modularity, maintainability, and clarity.

1. Layered Architecture Pattern
The backend was implemented using a layered structure consisting of Controllers, Services, and Repositories. Controllers handle HTTP requests and responses, Services contain the core business logic, and Repositories manage persistence. This separation improves maintainability and ensures that each layer has a clear responsibility.

2. Repository Pattern
The Repository pattern was used through Spring Data JPA repositories such as UserRepository, ItemRepository, BidRepository, and PaymentRepository. This pattern abstracts database access from the service layer and allows persistence logic to remain separated from business logic.

3. DTO (Data Transfer Object) Pattern
DTOs were used to separate the internal persistence model from the API request and response structures. Request objects such as SignupRequest, LoginRequest, BidRequest, and PaymentRequest, as well as response objects such as LoginResponse, PaymentResponse, and ReceiptResponse, help isolate external API contracts from internal entities.

4. Dependency Injection Pattern
Dependency Injection was used extensively through Spring Boot to inject services and repositories into controllers and service classes. This reduces coupling between classes and improves testability, maintainability, and flexibility of the application.

Design Patterns Considered but Not Used

1. Observer Pattern

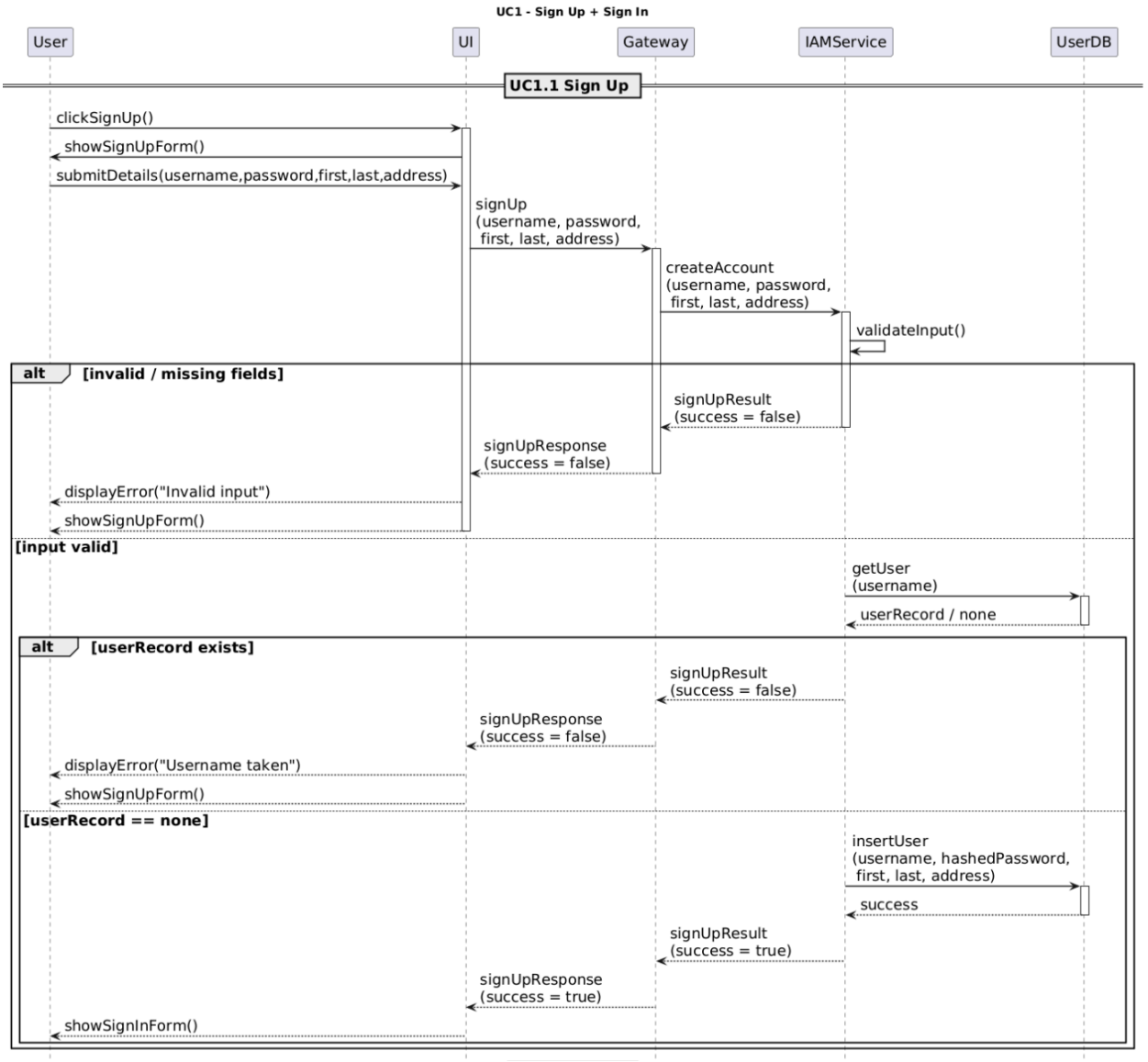
The Observer pattern was considered for supporting real-time auction notifications, such as notifying users when an auction ends or when a new highest bid is placed. However, this was not implemented in Deliverable 2 because asynchronous notifications were not required for this milestone.

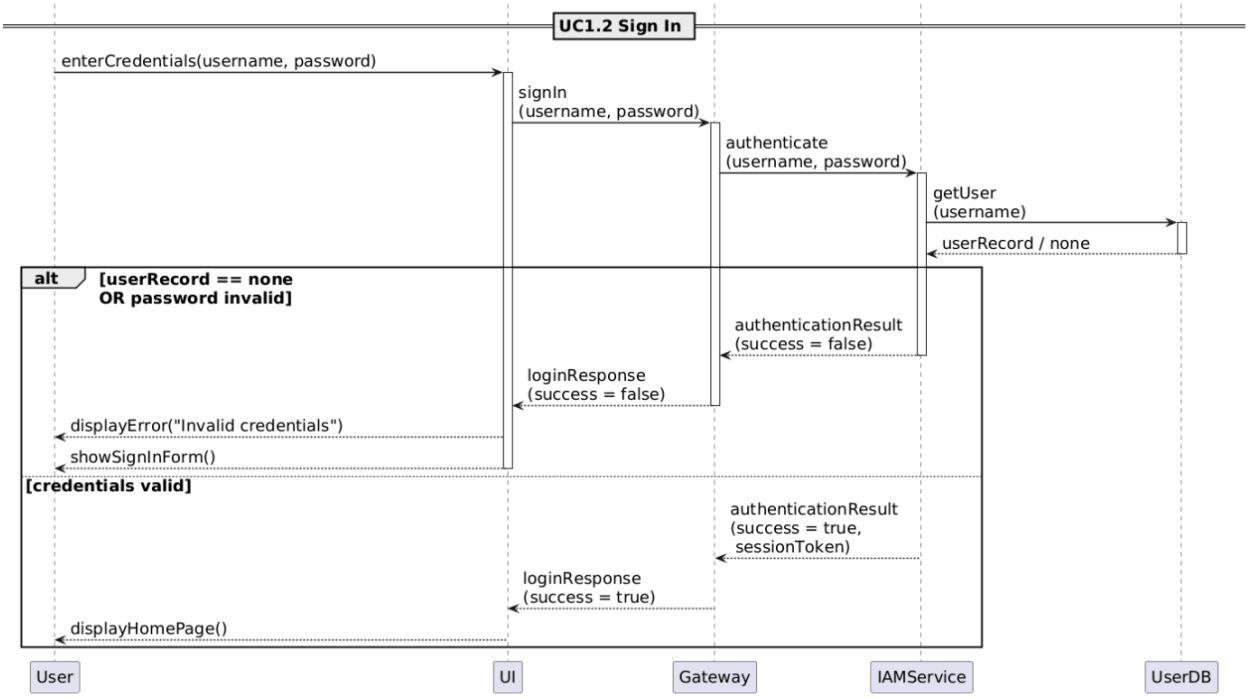
2. Singleton Pattern

The Singleton pattern was considered for services managing application-wide state. However, Spring Boot already manages service and repository beans with singleton scope by default, so implementing a separate Singleton pattern was unnecessary.

3 Sequence Diagrams

UC1: Sign Up and Sign In



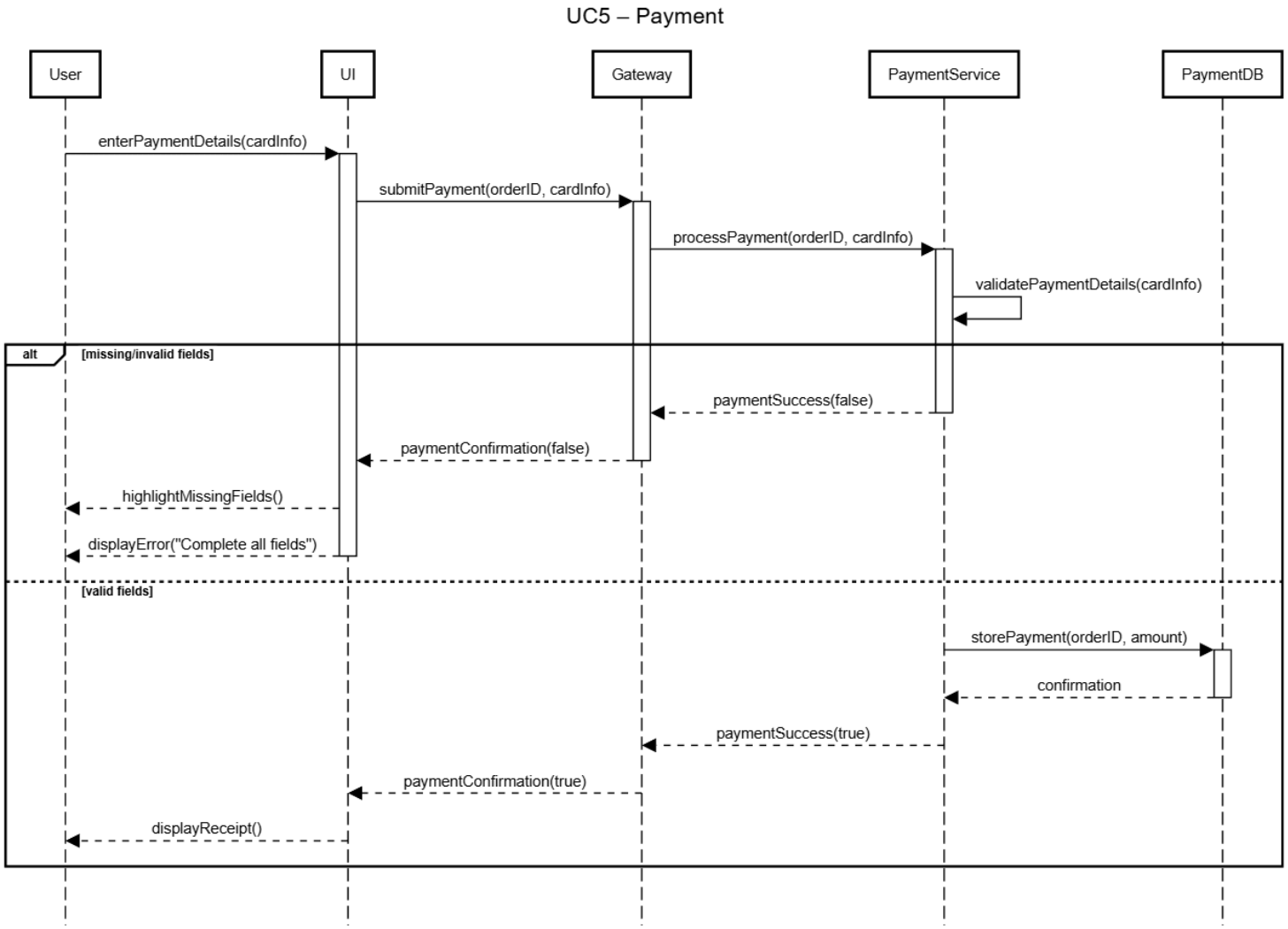


This sequence diagram models the complete user authentication workflow, including both sign-up (UC1.1) and sign-in (UC1.2). During sign-up, the user submits registration details through the UI, which sends requests to the Gateway and then to the IAMService. The IAMService validates input, checks for existing users in the UserDB, and either rejects the request or creates a new user record. Upon successful registration, the user is redirected to the sign-in form.

For sign-in, the user submits credentials that are routed through the Gateway to the IAMService for authentication. The IAMService retrieves user data from the UserDB and verifies the credentials. If authentication fails, an error message is displayed and the user is prompted to retry. If authentication succeeds, a session token is generated and the user is granted access to the home page.

For additional clarity, a high-resolution version of the same diagram has also been uploaded to Google Drive to allow reviewers to view the diagram at full scale if needed using the following link: Here!

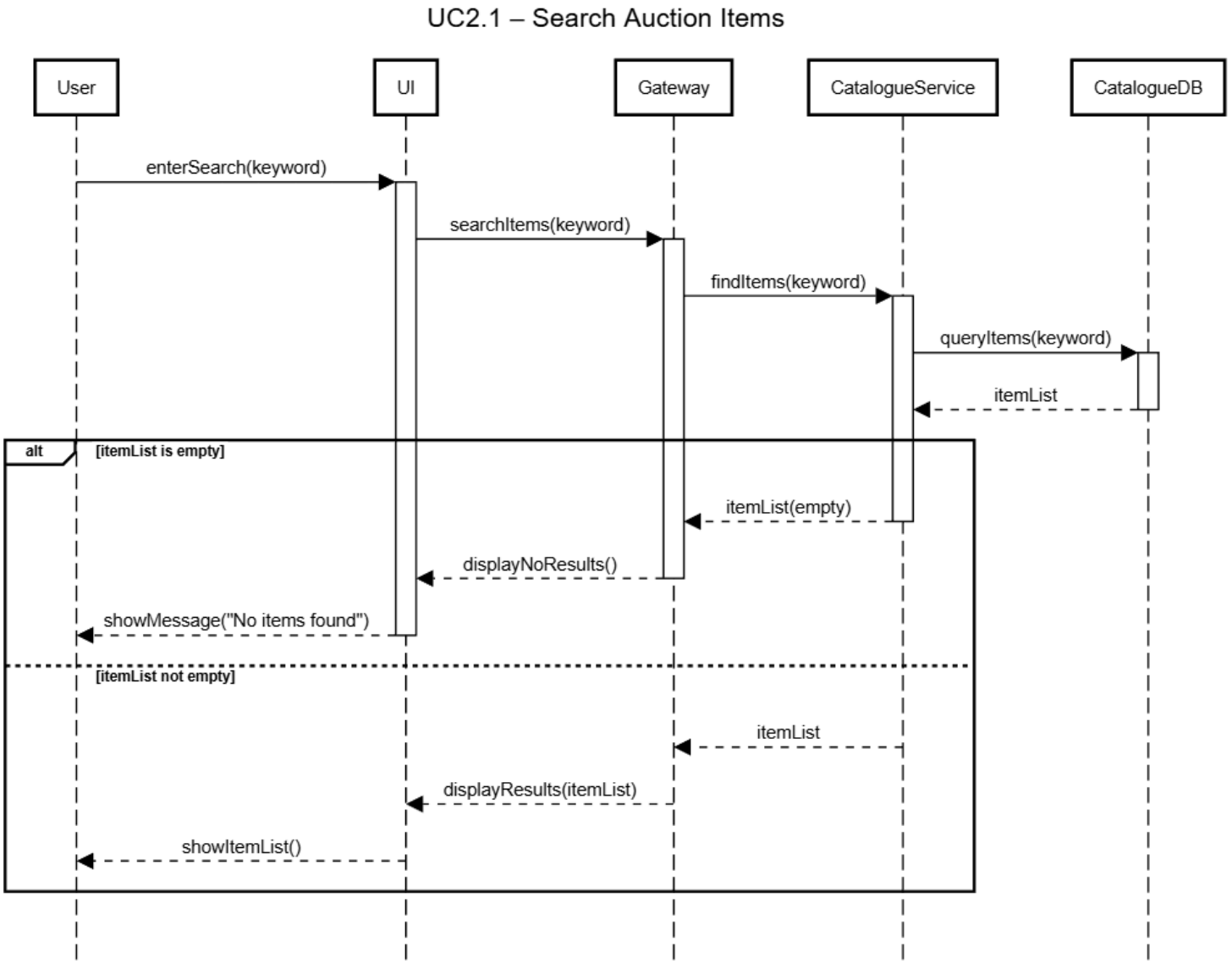
UC5: Payment



This diagram depicts the payment process after an auction has ended. The user enters payment details through the UI, which forwards the request to the Gateway and then to the PaymentService. The PaymentService first validates the payment details.

If required fields are missing or invalid, the payment is rejected and the UI highlights the missing information and displays an error message. If the payment details are valid, the transaction is stored in the PaymentDB and a confirmation is returned. The UI then displays the payment receipt to the user, completing the payment workflow.

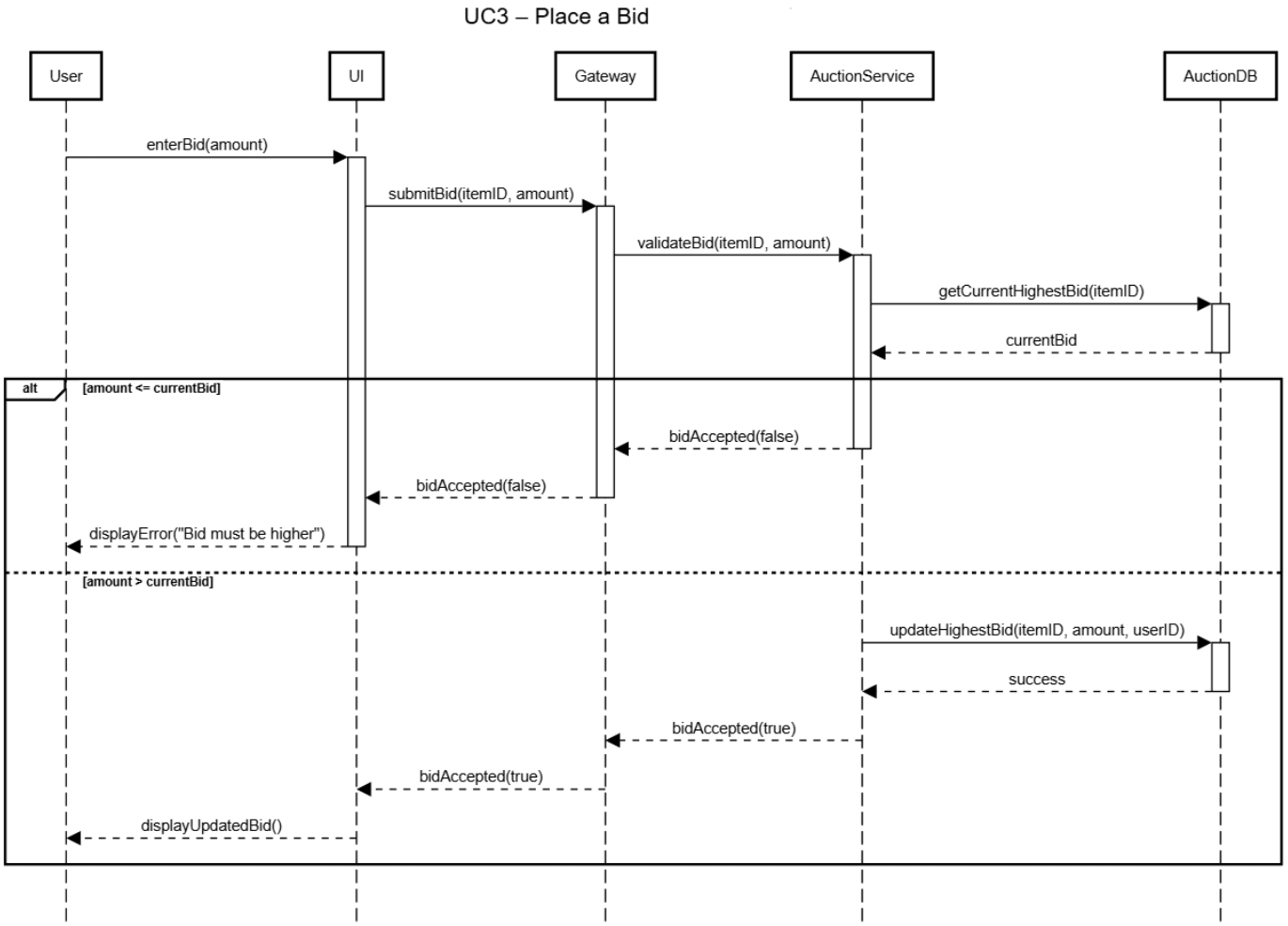
UC2.1: Search Auction Items



This diagram illustrates the auction item search process. The user enters a search keyword through the UI, which sends the request to the Gateway. The Gateway delegates the request to the CatalogueService, which queries the CatalogueDB for matching auction items.

Two outcomes are modeled: if no items match the search criteria, an empty result set is returned and the UI displays a “No items found” message. If matching items exist, the item list is returned to the UI and displayed to the user. This diagram demonstrates proper separation of concerns, where the UI handles presentation, services handle business logic, and the database manages persistence.

UC3: Place a Bid

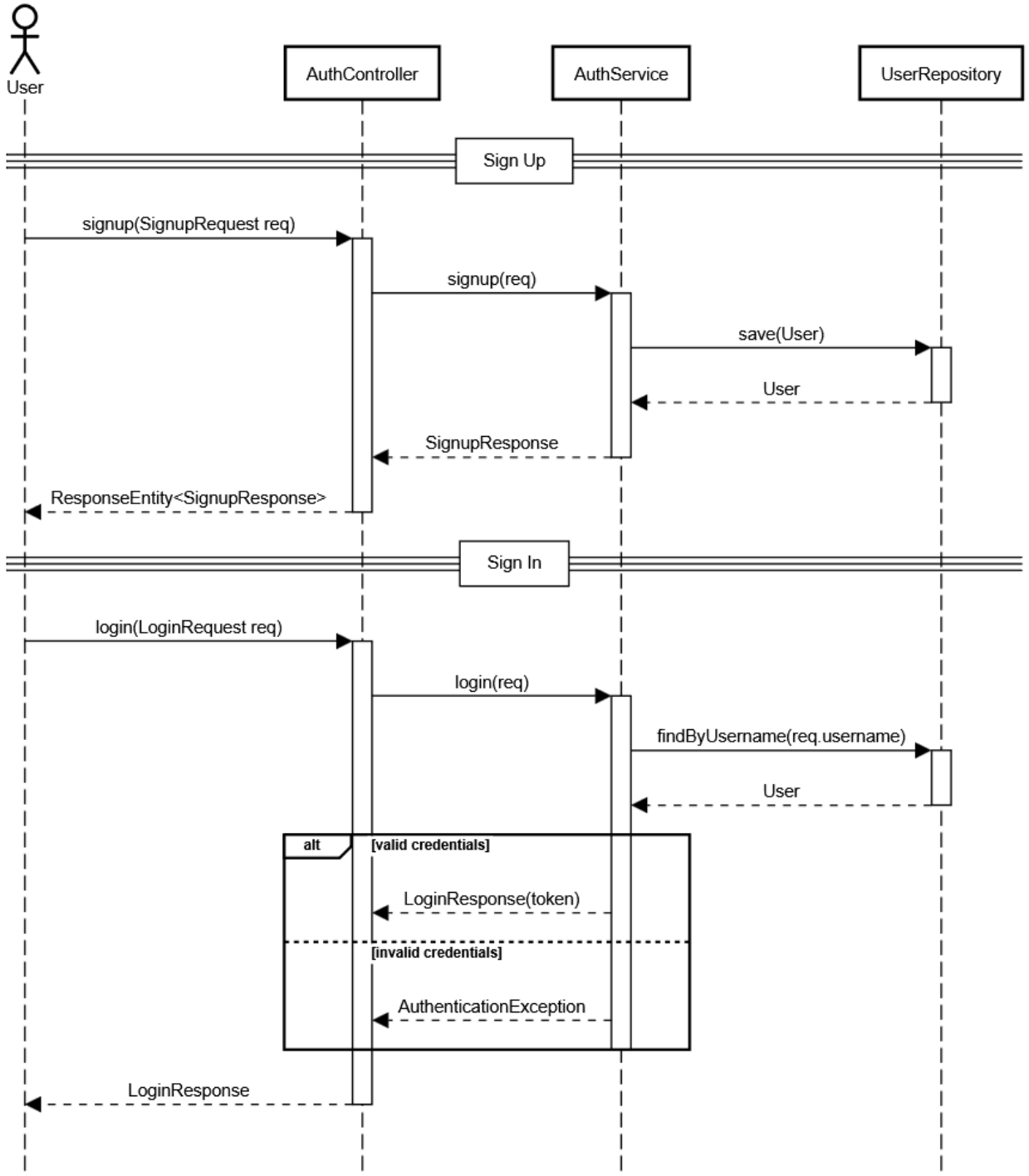


This sequence diagram represents the bidding process for a forward auction. The user submits a bid amount via the UI, which sends the request to the Gateway and subsequently to the AuctionService. The AuctionService retrieves the current highest bid from the AuctionDB and validates the new bid amount.

If the submitted bid is less than or equal to the current highest bid, the bid is rejected and an error message is displayed to the user. If the bid is valid, the AuctionService updates the highest bid in the database and returns a success response. The updated bid information is then displayed to the user through the UI.

3 (Appended) Sequence Diagrams

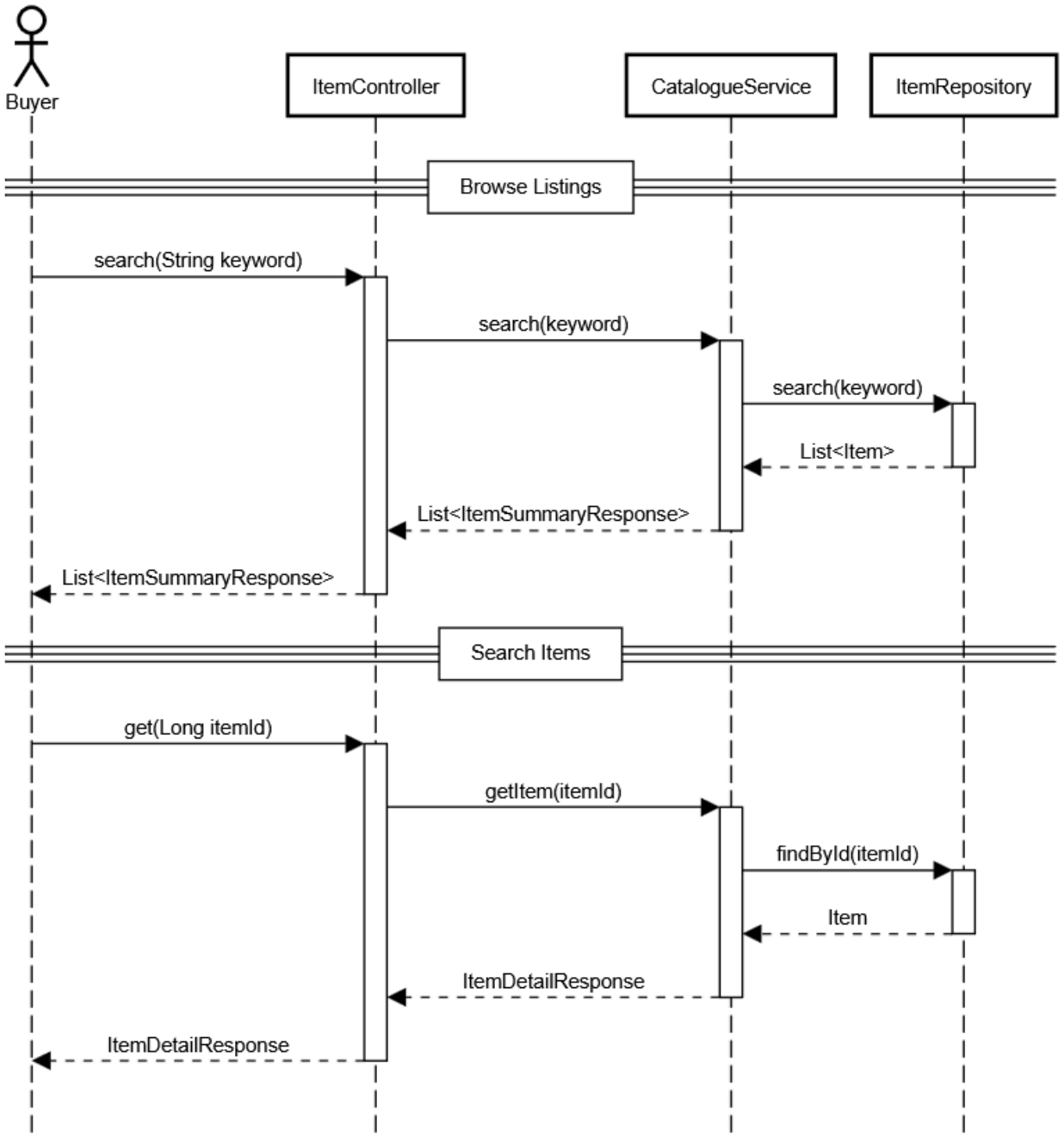
UC1: Sign-up and Login



This sequence diagram models the user authentication workflow, including both account creation and login. During sign-up, the user sends a registration request to the AuthController, which forwards the request to the AuthService. The service validates the input and stores the new user by calling UserRepository.save(). If the registration is successful, a response confirming the account creation is returned to the user.

For login, the user submits their credentials through the AuthController. The controller calls AuthService.login(), which retrieves the user from the database using UserRepository.findByUsername(). The service verifies the credentials and determines whether authentication succeeds or fails. If the credentials are valid, a LoginResponse is returned and the user is authenticated; otherwise, an error message is returned.

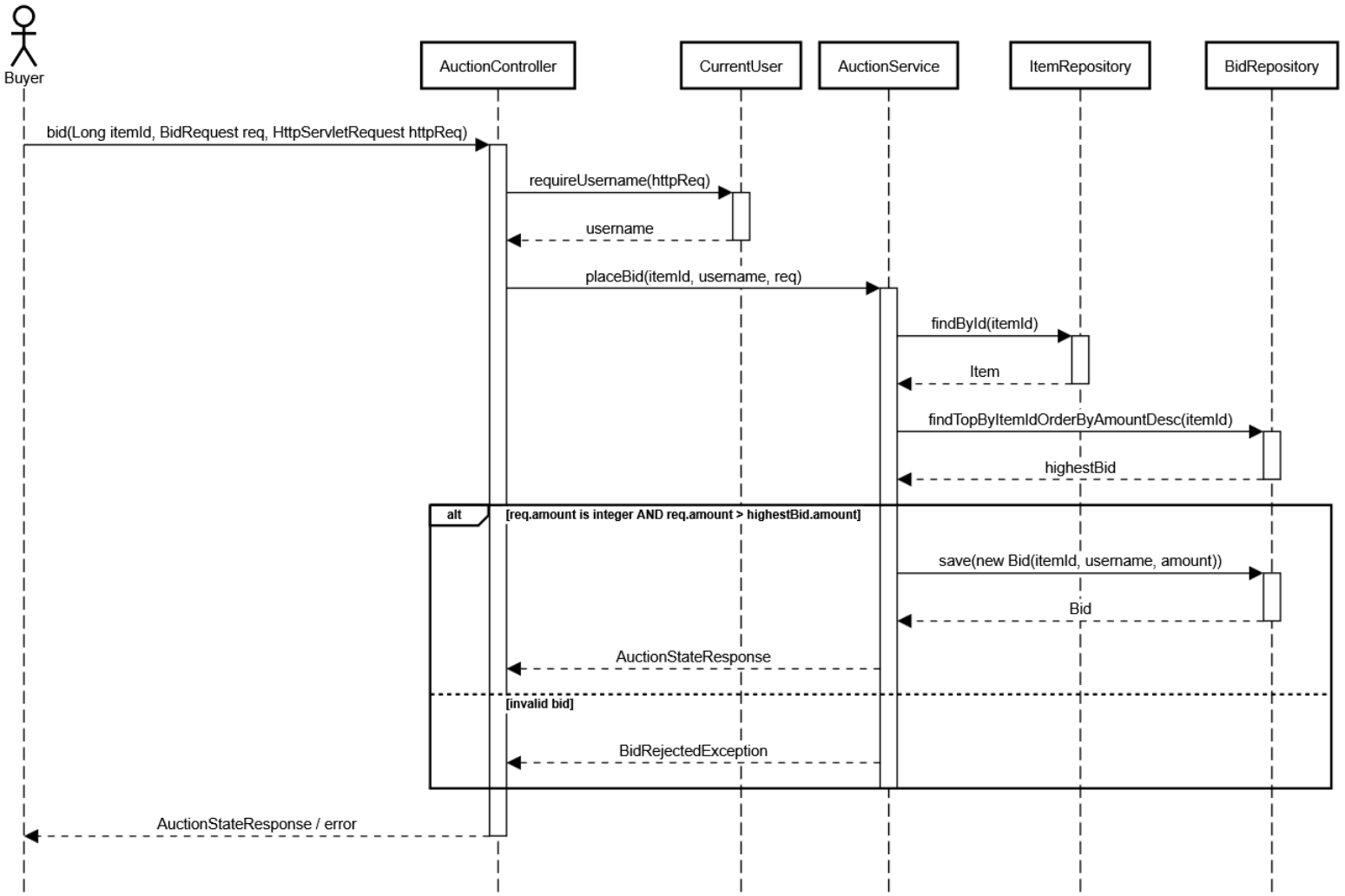
UC2: Browse and Search Catalogue



This sequence diagram models how buyers browse or search for items in the auction catalogue. When a buyer searches for items, a request is sent to the ItemController, which forwards the query to the CatalogueService. The service calls ItemRepository.search() to retrieve matching items from the database and converts them into response objects for the buyer.

If the buyer selects a specific item, the ItemController calls CatalogueService.getItem() to retrieve detailed item information. The service fetches the item using ItemRepository.findById(). The resulting item details are returned to the controller and displayed to the buyer.

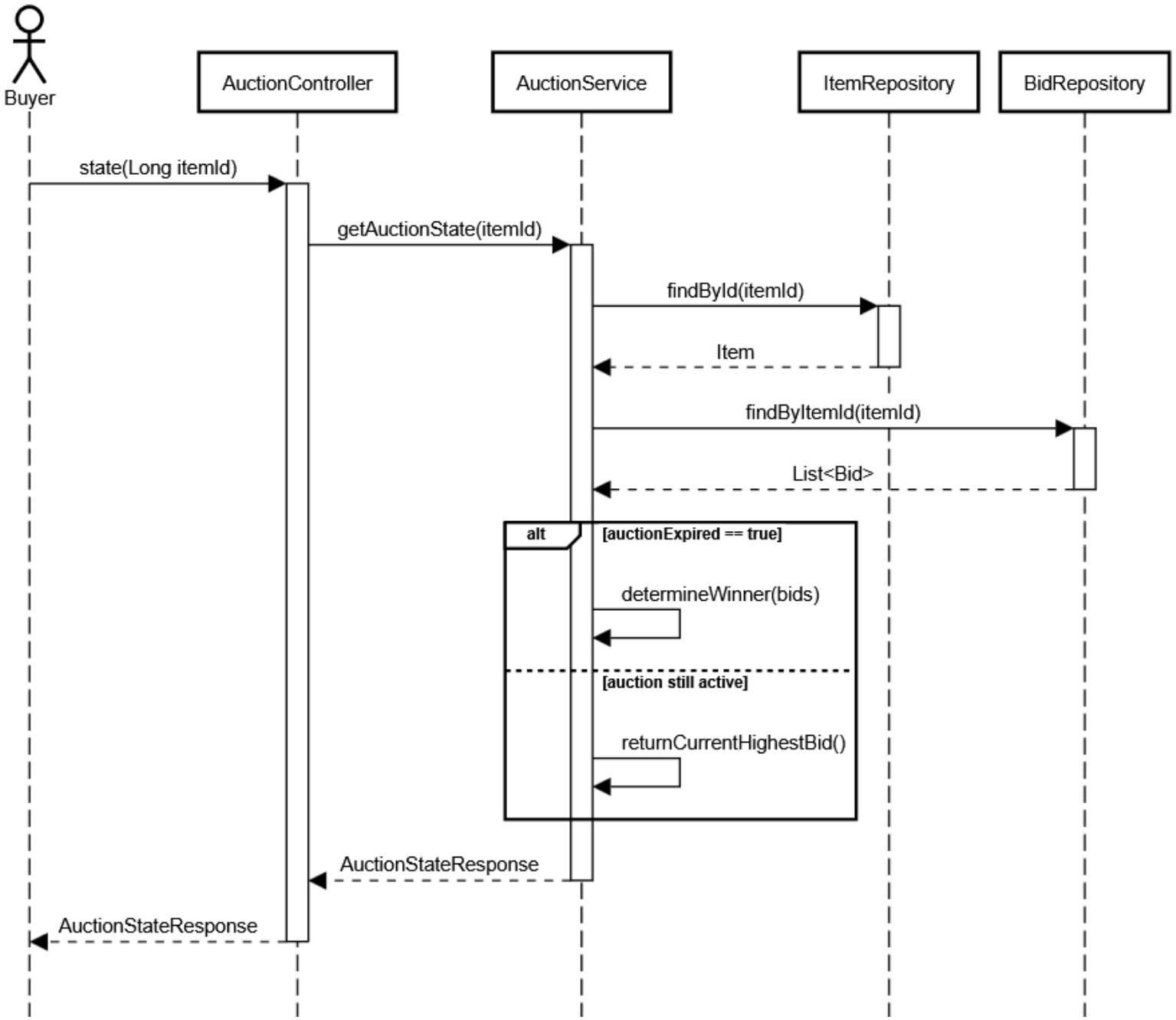
UC3: Placing and Validating Bids



This sequence diagram models the process of placing a valid bid during an auction. The buyer submits a bid request to the AuctionController, which first identifies the logged-in user using CurrentUser.requireUsername(). The controller then calls AuctionService.placeBid() to process the bid.

The service retrieves the item and the current highest bid using ItemRepository.findById() and BidRepository.findTopByItemIdOrderByAmountDesc(). The bid is validated to ensure the amount is an integer and greater than the current highest bid. If the bid is valid, it is saved using BidRepository.save() and the updated auction state is returned. If the bid is invalid, the request is rejected and an error response is returned.

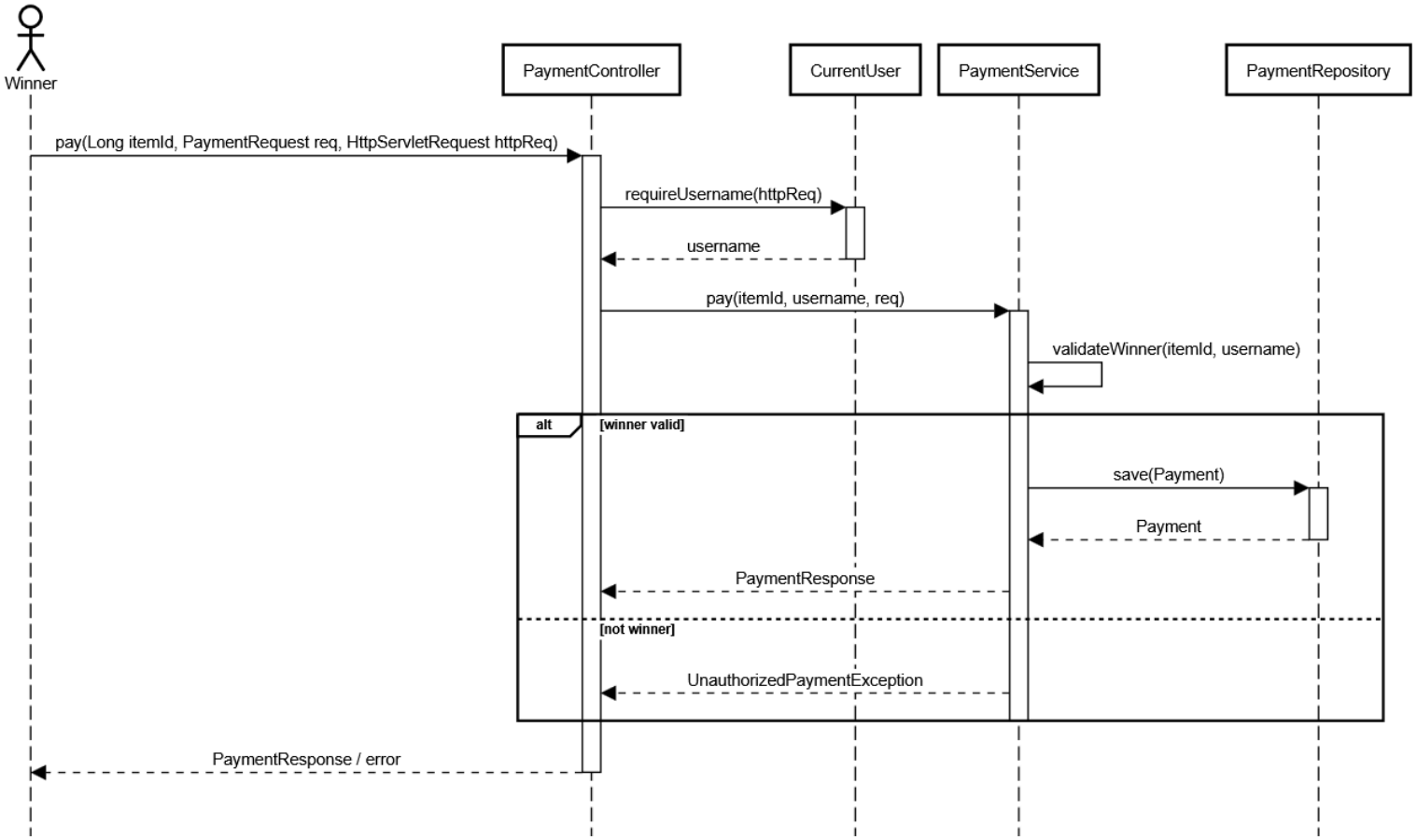
UC4: Auction End Behaviour



This sequence diagram models how the system determines the state of an auction and identifies the winner when the auction ends. A buyer requests the auction status through the AuctionController, which calls AuctionService.getAuctionState().

The service retrieves the item and its associated bids using ItemRepository.findById() and BidRepository.findByItemId(). It then checks whether the auction has expired. If the auction has ended, the highest bid is selected as the winning bid; otherwise, the current auction state is returned. The resulting auction information is sent back to the buyer.

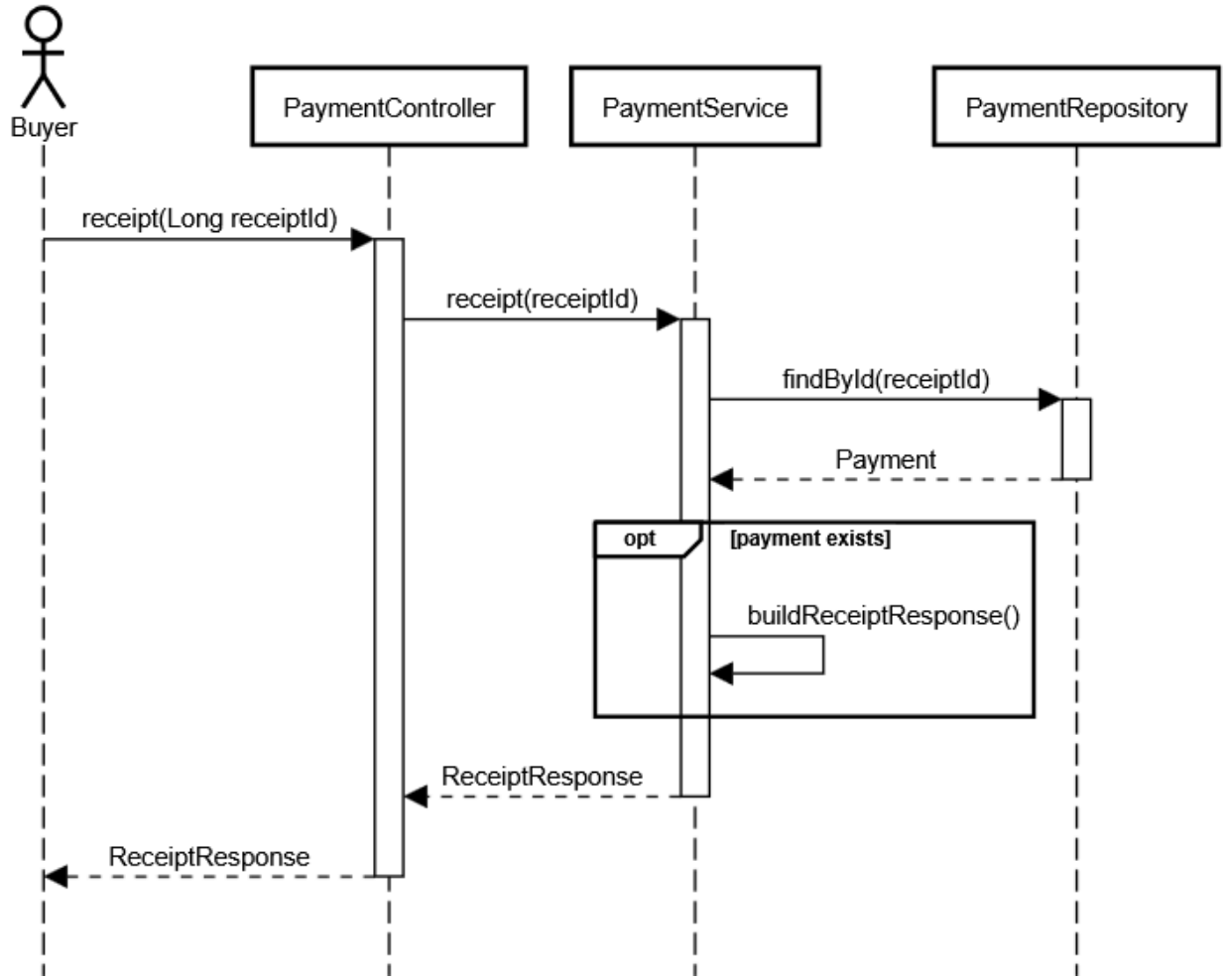
UC5: Payment



This sequence diagram models how the system processes payment for an auction item. The winning bidder submits a payment request to the PaymentController, which identifies the authenticated user using CurrentUser.requireUsername(). The controller then calls PaymentService.pay() to handle the payment.

The service verifies that the user attempting to pay is the winner of the auction. If the validation succeeds, a payment record is stored using PaymentRepository.save(). A PaymentResponse confirming the transaction is then returned to the user. If the user is not the winning bidder, the payment request is rejected.

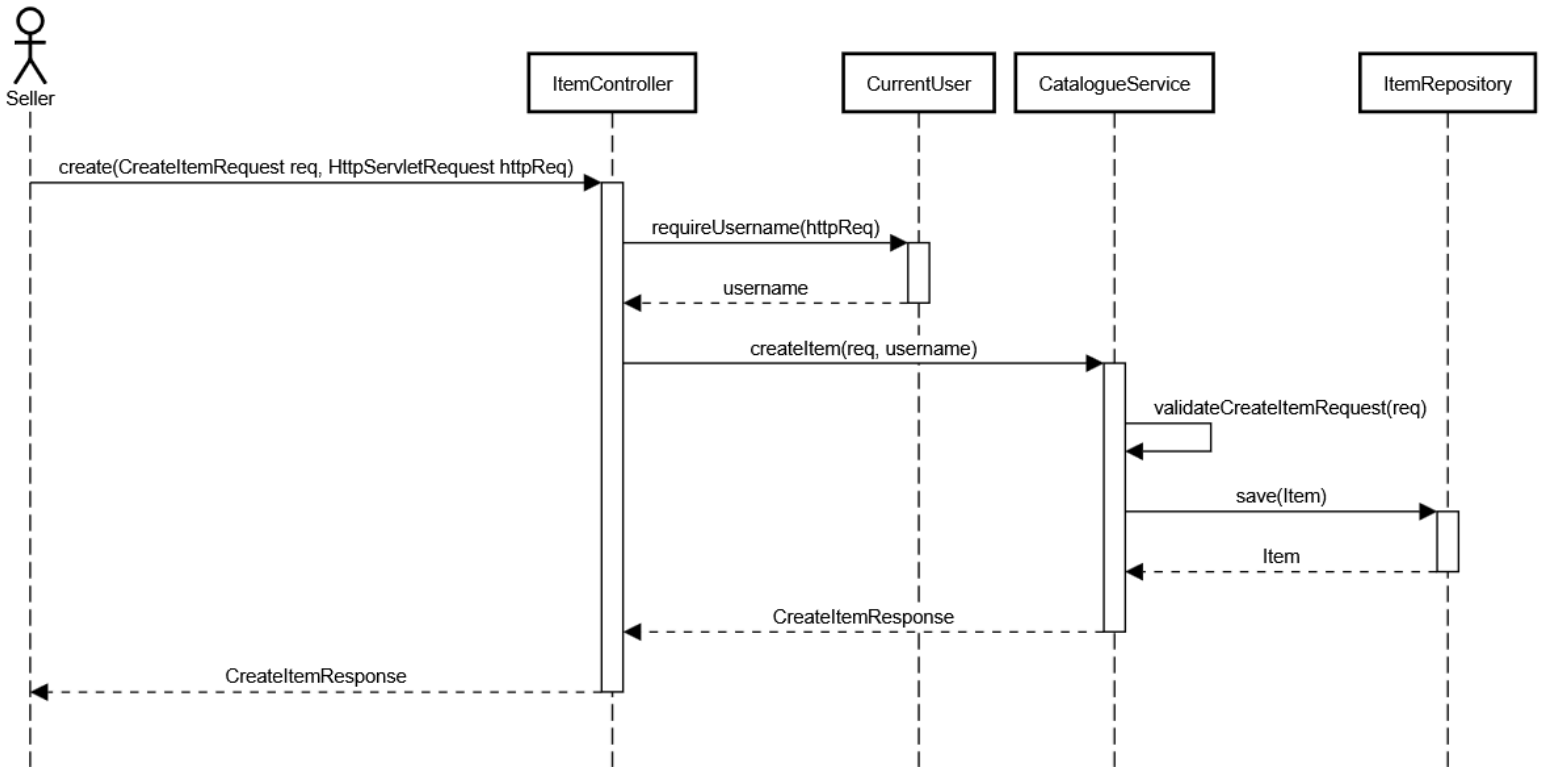
UC6: Receipt and Shipping



This sequence diagram models how the system generates a receipt after a payment is completed. The buyer requests a receipt through the PaymentController using a receipt identifier. The controller forwards the request to the PaymentService.

The service retrieves the payment information using PaymentRepository.findById(). It then constructs a ReceiptResponse containing the transaction details. The receipt is returned to the controller and displayed to the buyer as confirmation of the purchase.

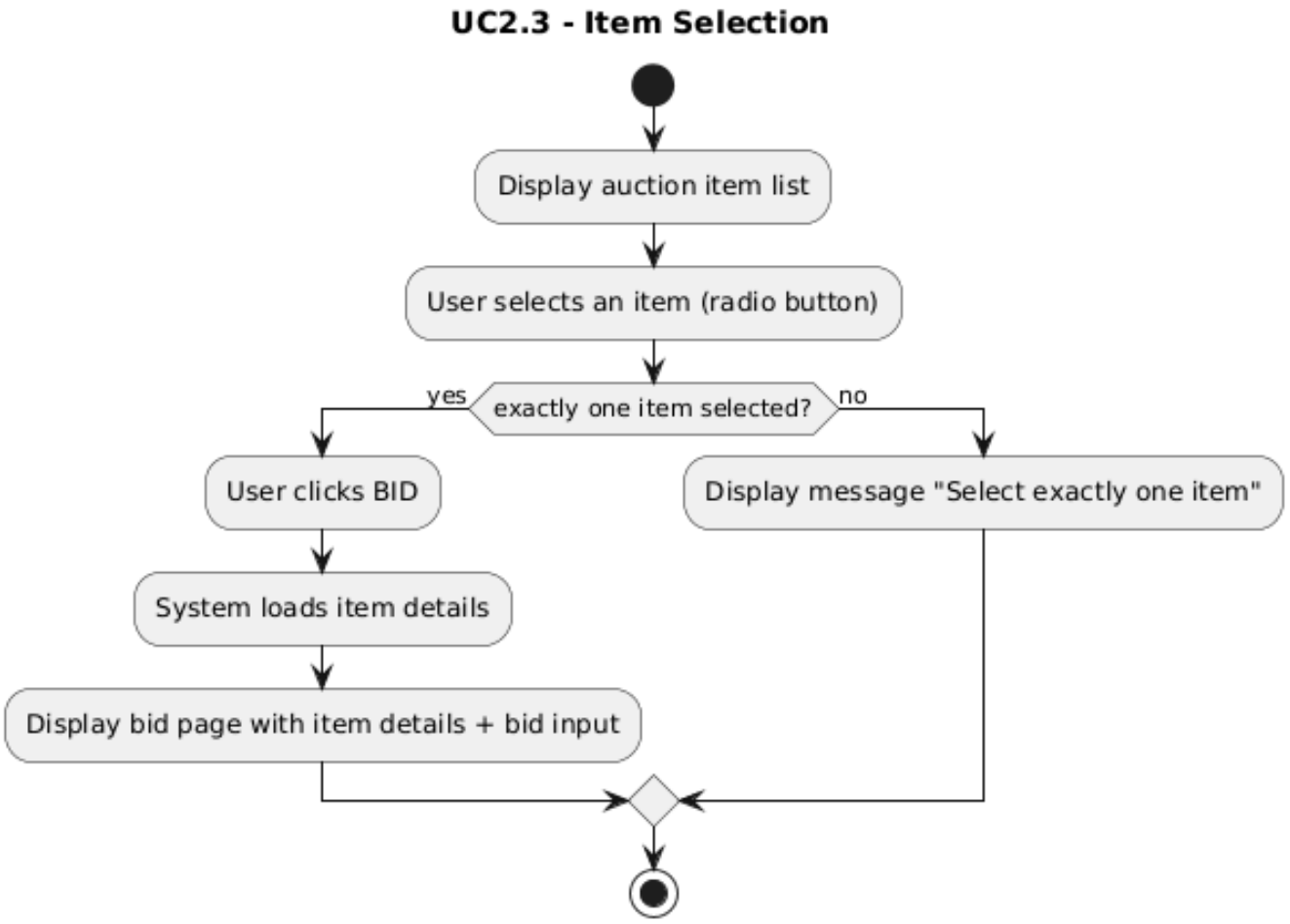
UC7: Creating An Auction Listing



This sequence diagram models the process of creating a new auction listing. The seller submits an item creation request to the ItemController. The controller identifies the authenticated seller using CurrentUser.requireUsername() and forwards the request to CatalogueService.createItem().

The service validates the item information and creates an Item object. The item is stored in the database using ItemRepository.save(). Once the item is successfully saved, a CreateItemResponse is returned to the seller confirming that the listing has been created.

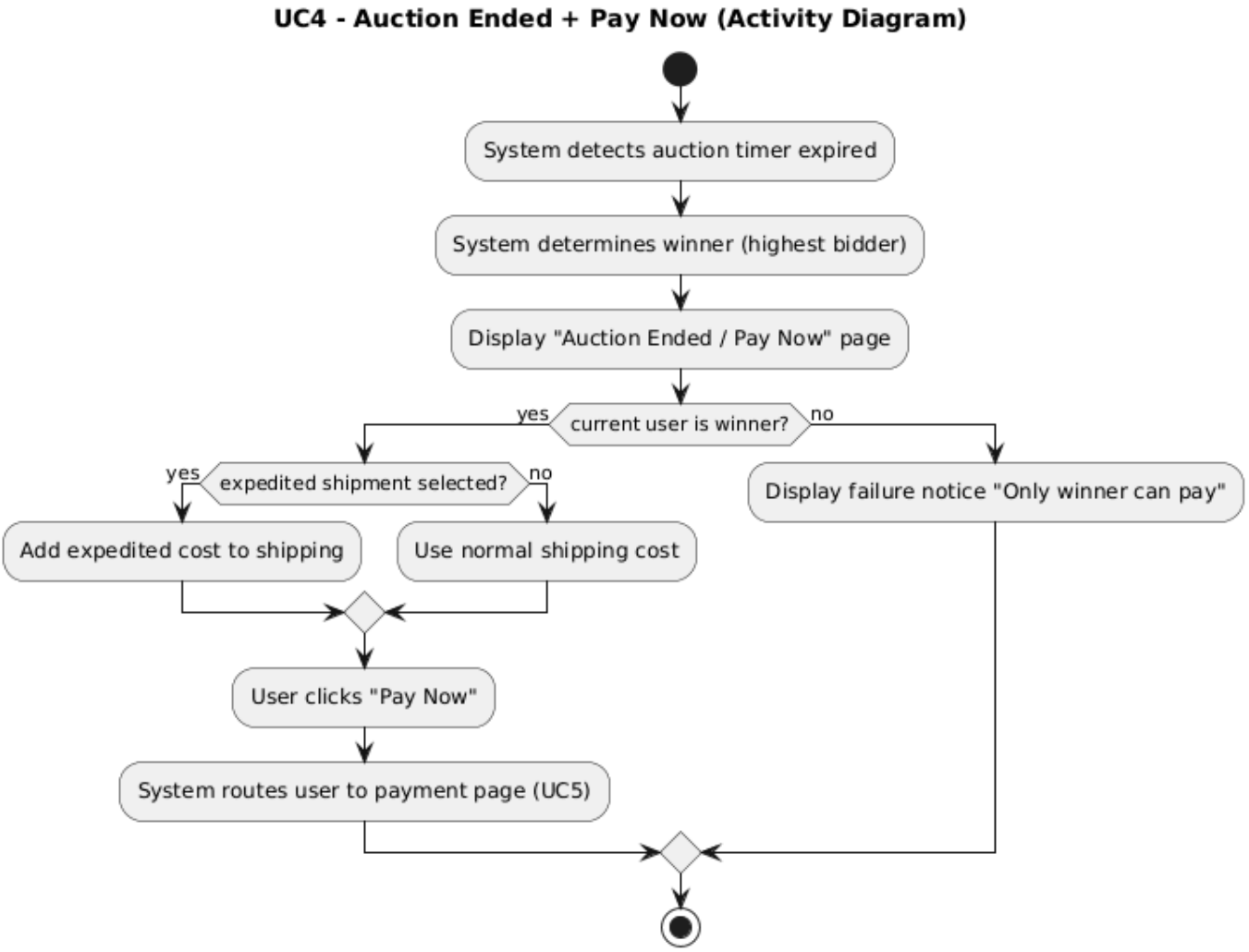
4 Activity Diagrams



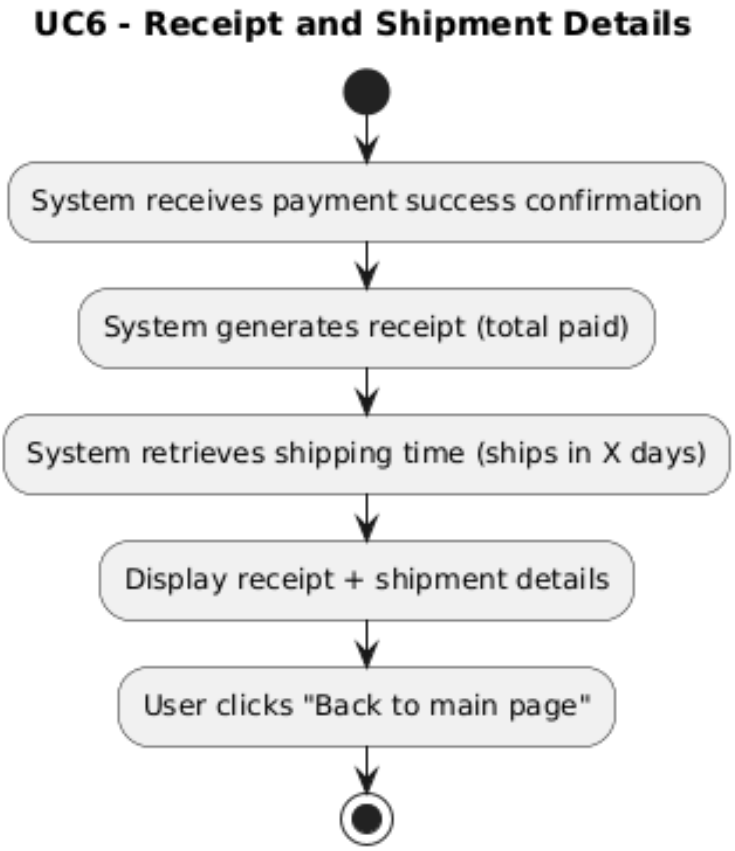
Legend for all Activity diagrams :

- Actions are rounded rectangles; diamonds represent decisions with guards; arrows represent control flow; start/end nodes indicate workflow boundaries.

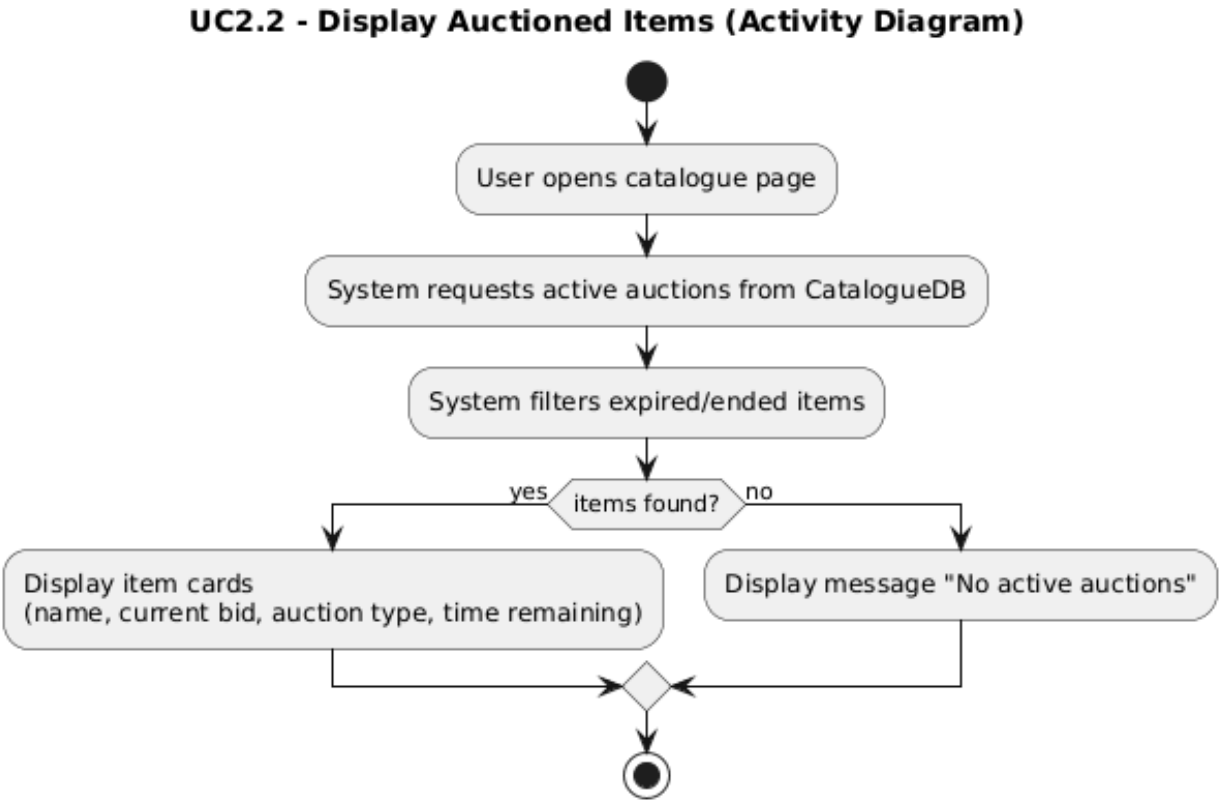
This diagram models how a user selects exactly one item to bid on. After the system displays the auction item list, the user selects an item using a radio button. A decision checks if exactly one item is selected. If not, the user is prompted to select exactly one item; otherwise the user proceeds by clicking BID, and the system loads the item details and displays the bid page.



This activity diagram captures the system behavior when an auction ends. After the timer expires, the system determines the winner and displays the “Auction Ended / Pay Now” page. A decision checks whether the current user is the winner; non-winners receive a failure notice and cannot proceed. Winners can optionally select expedited shipment, which updates the shipping cost, then click “Pay Now” to proceed to the payment page.



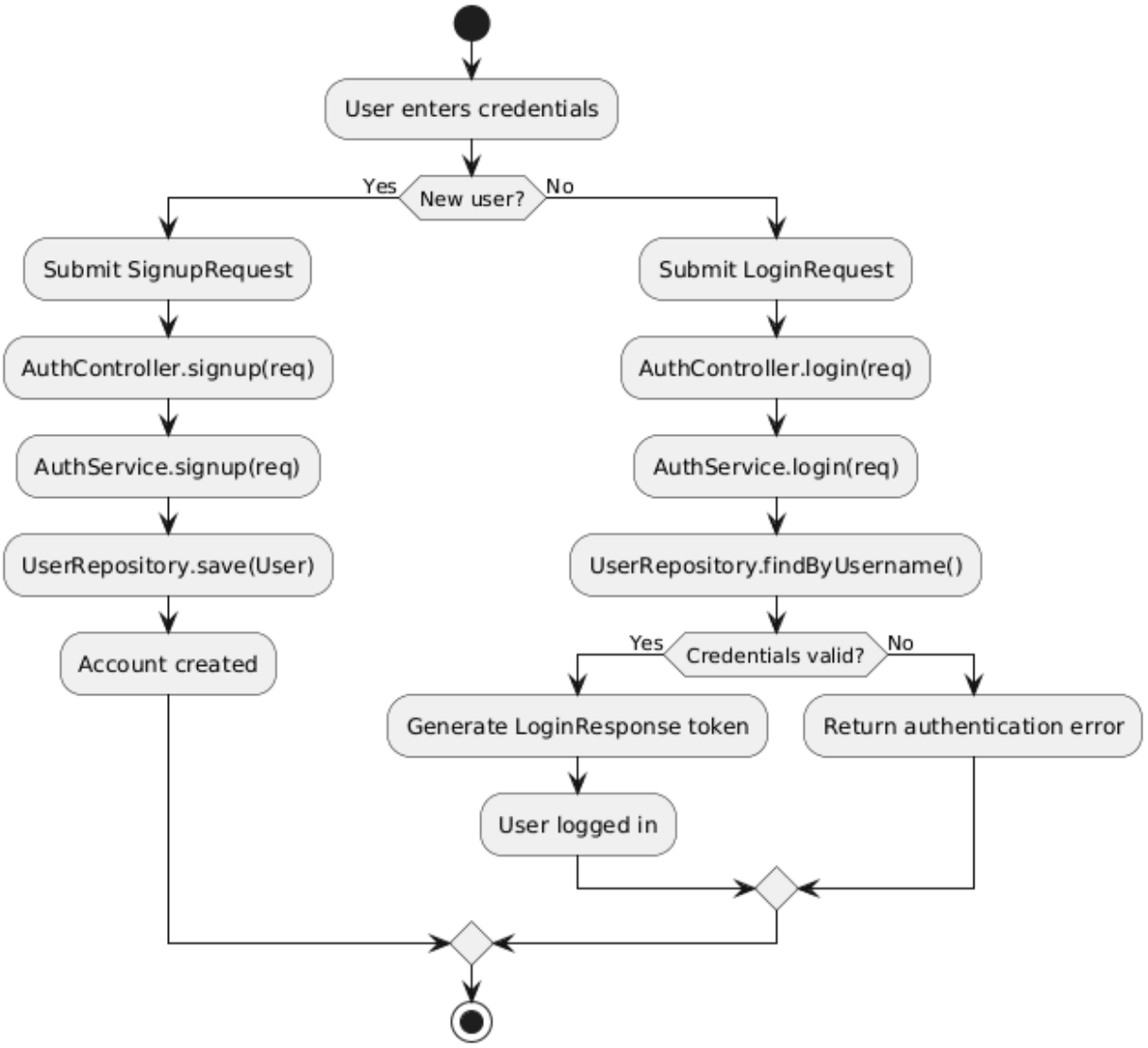
This diagram shows what happens after payment is confirmed. The system receives confirmation, generates a receipt containing the total paid, retrieves shipment timing , and displays both the receipt and shipping information. The user may then return to the main page.



This activity diagram shows how the system displays auction items after the user opens the catalogue page. The system requests active auctions, filters expired/ended items, and then branches based on whether items are found. If auctions exist, item cards are displayed; otherwise the user receives a “No active auctions” message.

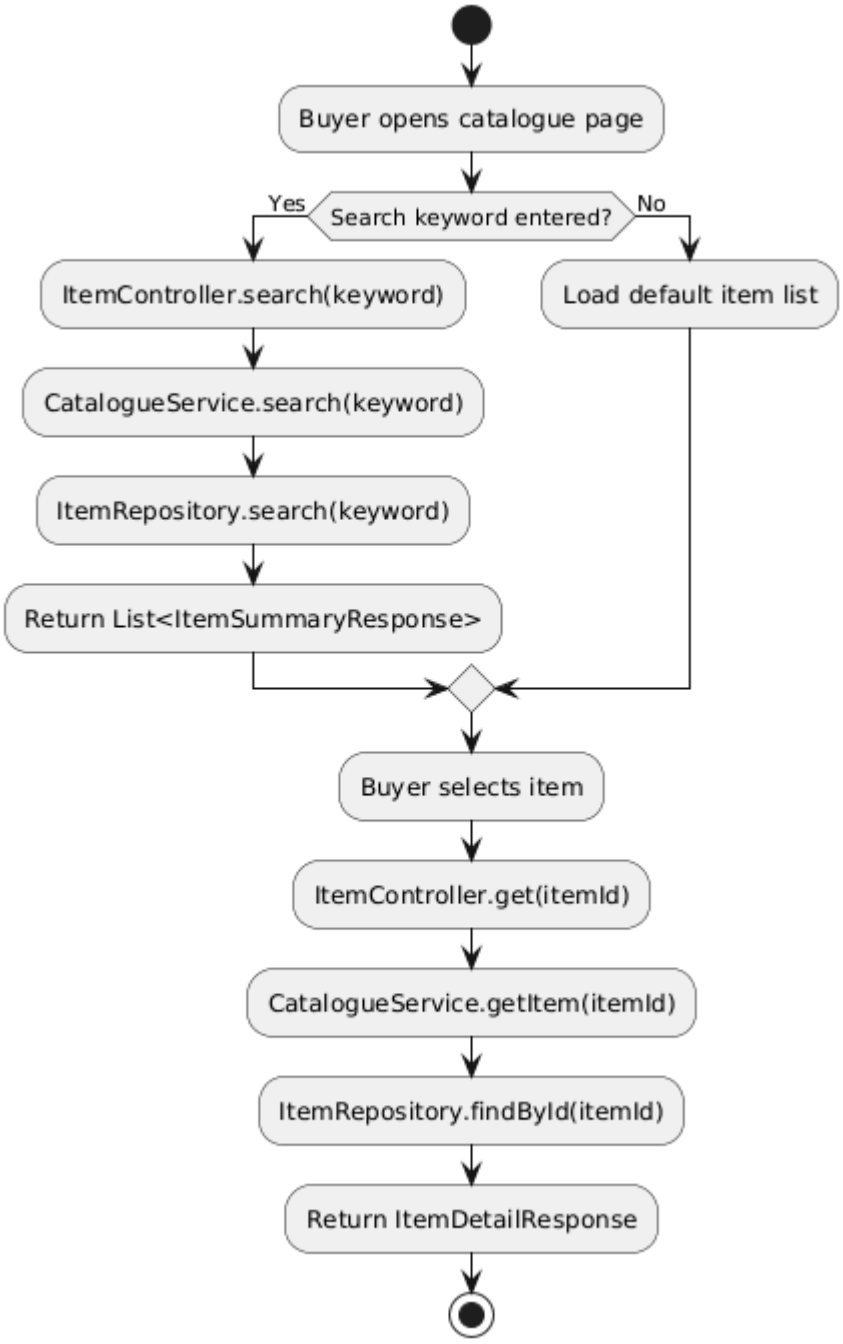
4 (Appended)Activity Diagrams

UC1: Sign-up and Login



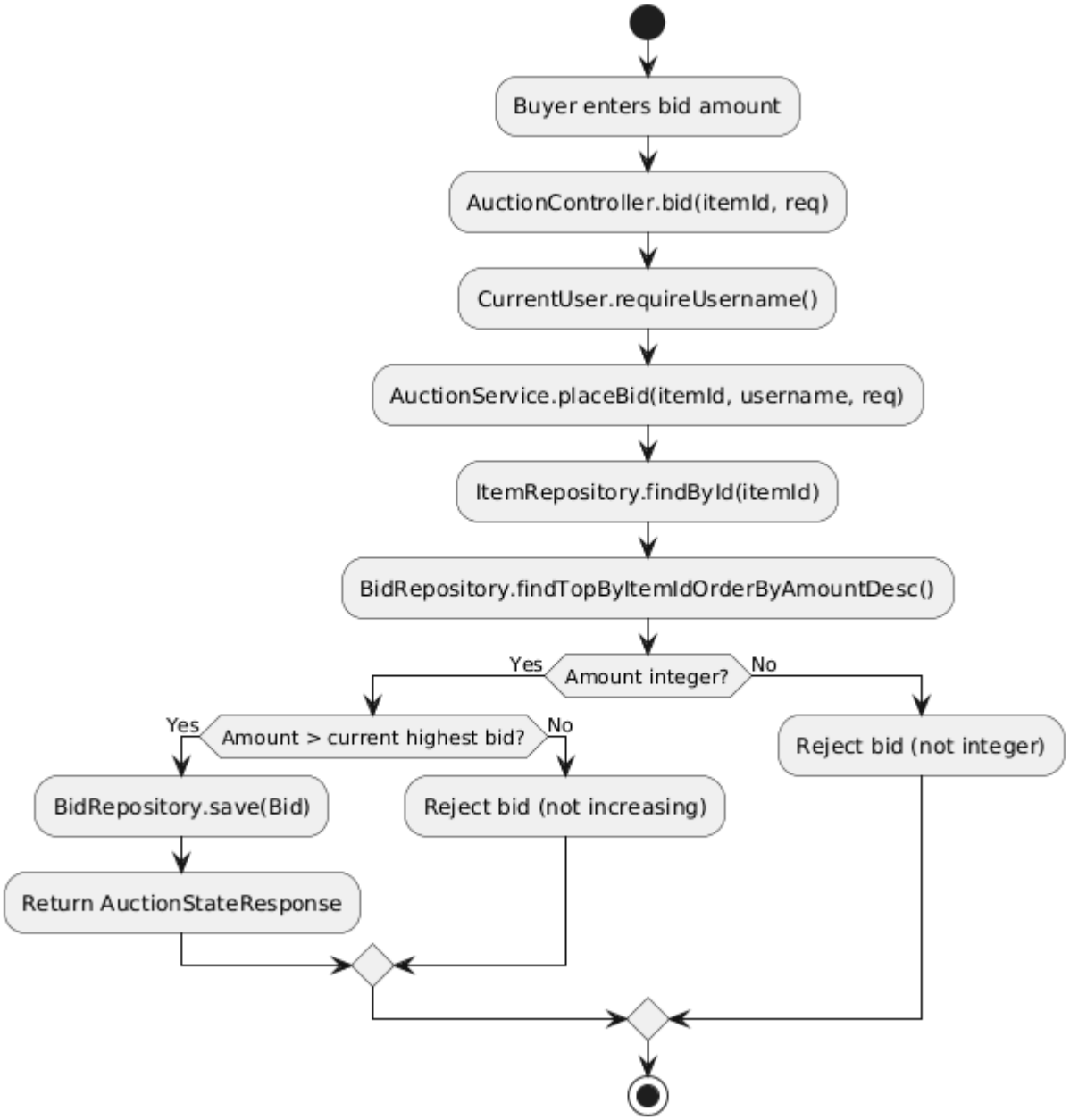
This activity diagram shows how the system handles user authentication for both new and returning users. The user submits credentials through the interface, and the system branches depending on whether the user is signing up or logging in. For sign-up, the system creates a new account in the database. For login, the system verifies the credentials and either grants access by generating a session token or returns an authentication error.

UC2: Browse and Search Catalogue



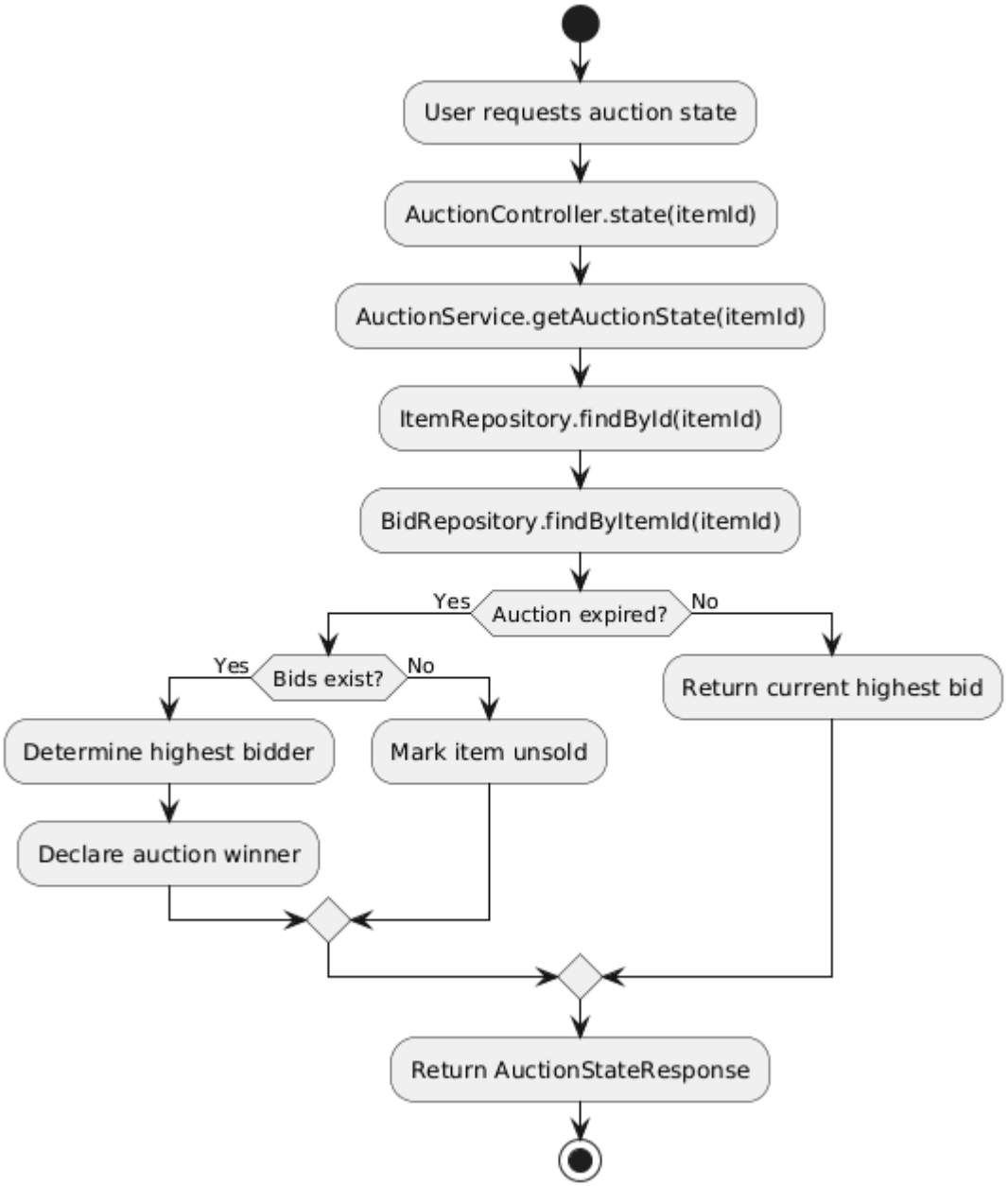
This activity diagram shows how the system displays auction items when a buyer browses the catalogue. The system first checks whether the user entered a search keyword. If a keyword is provided, the system searches the catalogue and retrieves matching items from the database. If no keyword is entered, the system loads the default list of items. The user can then select an item to view its detailed information.

UC3: Placing and Validating Bids



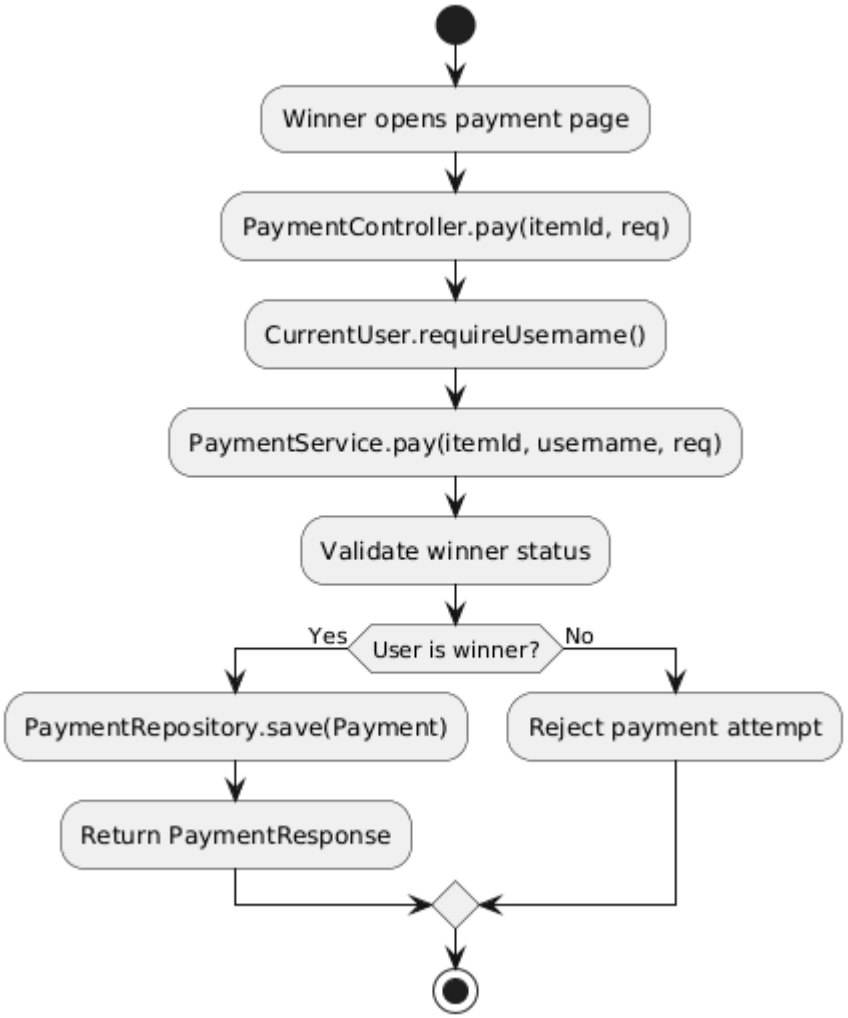
This activity diagram illustrates how the system processes a buyer's bid on an auction item. After the buyer submits a bid amount, the system retrieves the current highest bid and validates the new bid. The bid must be an integer and strictly greater than the existing highest bid. If both conditions are satisfied, the bid is saved and the auction state is updated. Otherwise, the system rejects the bid and informs the user of the validation error.

UC4: Auction End Behaviour



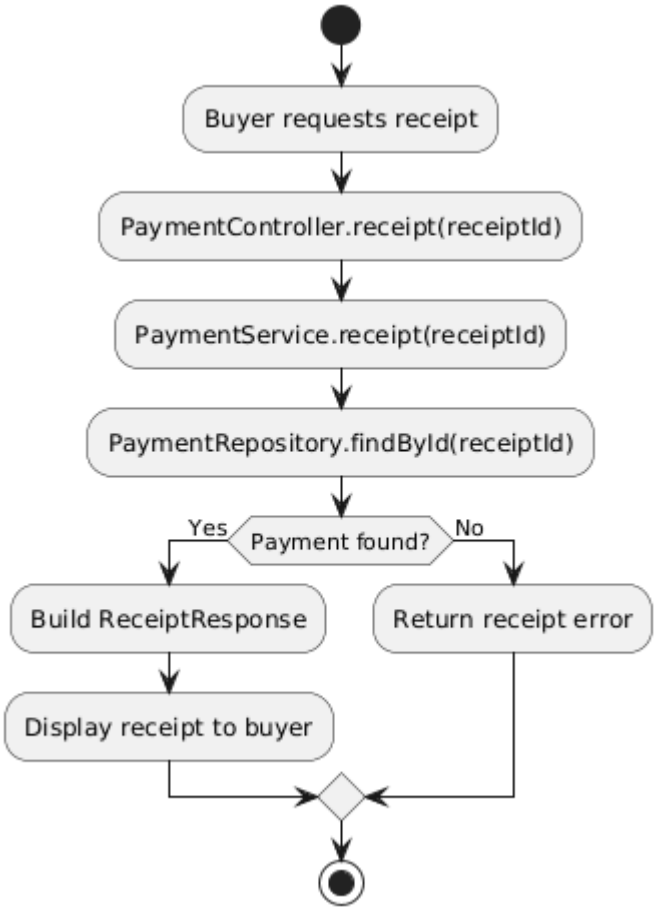
This activity diagram shows how the system determines the outcome of an auction when its state is requested. The system retrieves the item and its associated bids, then checks whether the auction has expired. If the auction has ended and bids exist, the system determines the highest bidder and declares the winner. If no bids exist, the item is marked as unsold. If the auction is still active, the current highest bid is returned.

UC5: Payment



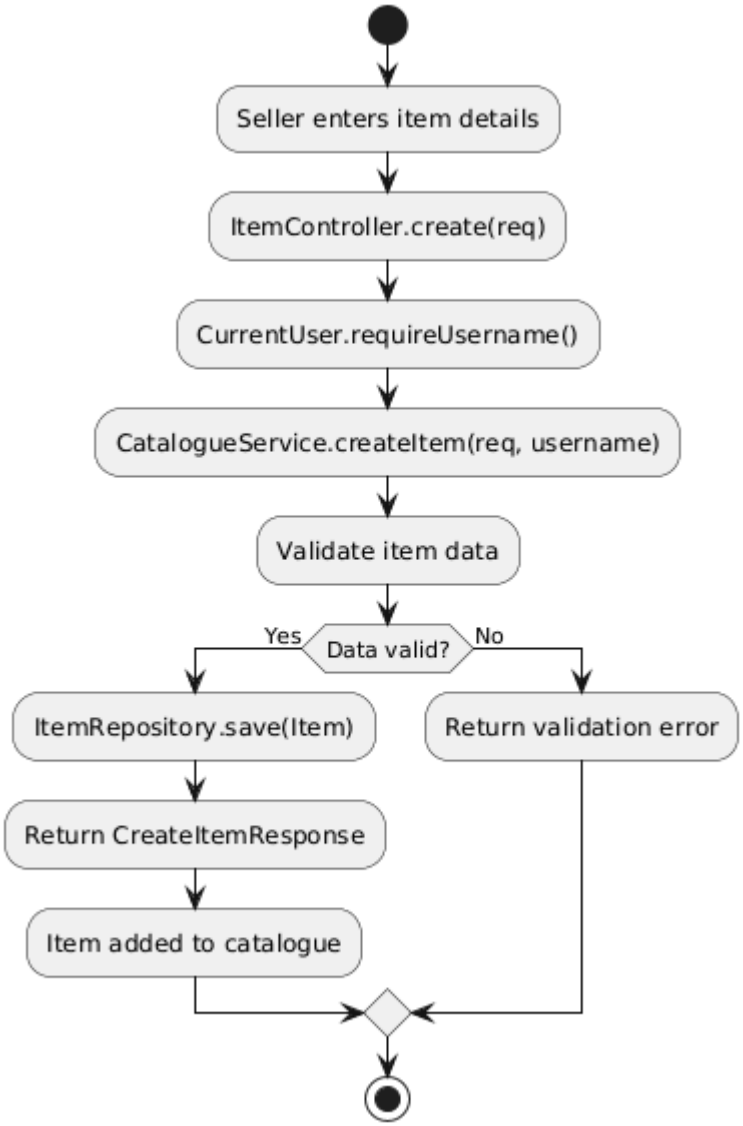
This activity diagram shows how the system processes payment for a completed auction. When the winning bidder submits payment information, the system verifies the user's identity and confirms that they are the winner of the auction. If the user is authorized, the payment is recorded in the database and a payment response is returned. If the user is not the winner, the system rejects the payment attempt.

UC6: Receipt and Shipping



This activity diagram illustrates how the system retrieves and displays a payment receipt. When the buyer requests the receipt, the system searches for the corresponding payment record. If the record exists, the system generates a receipt response containing the transaction details and displays it to the user. If the payment record cannot be found, the system returns an error message.

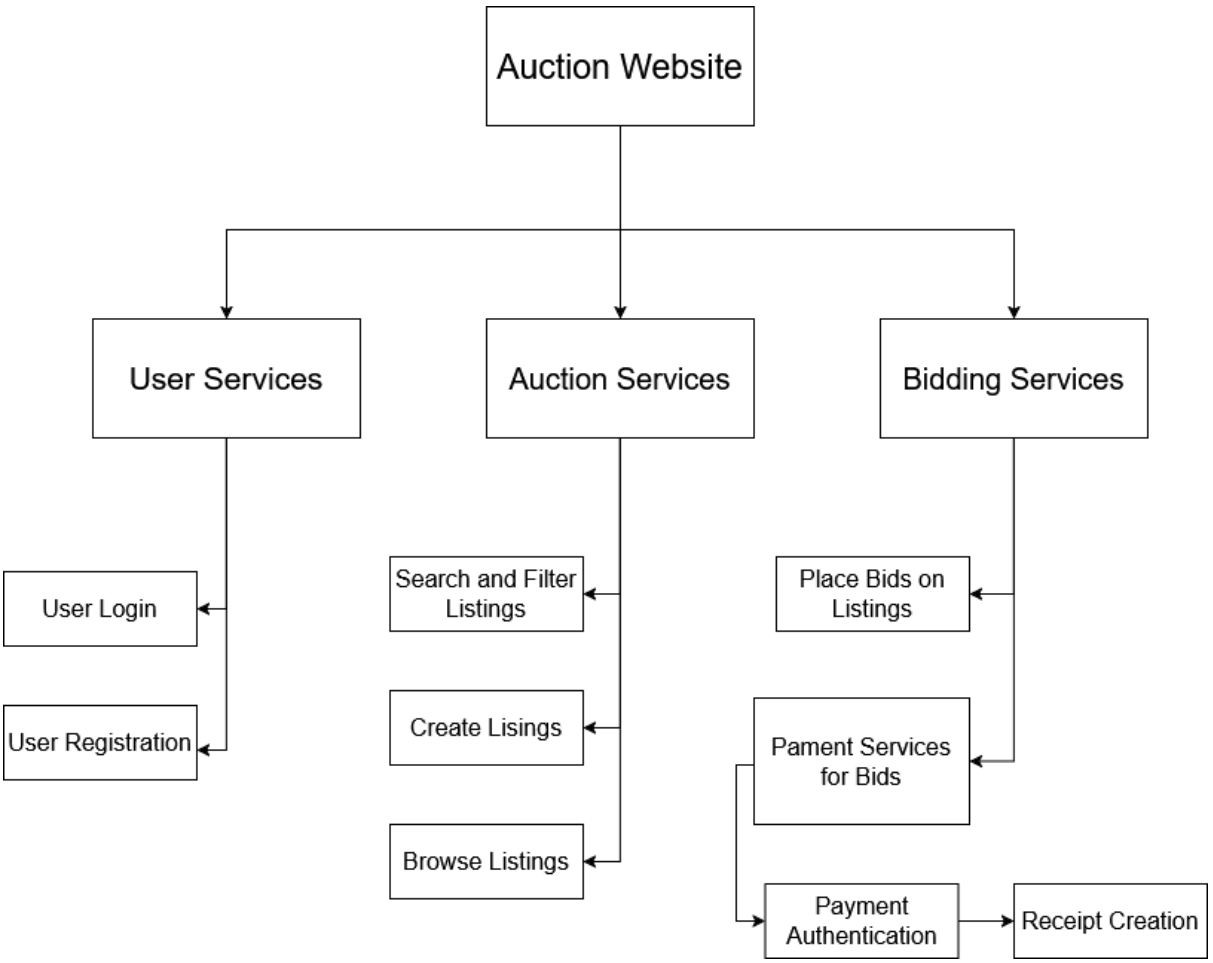
UC7: Creating An Auction Listing



This activity diagram shows how a seller uploads a new item to the auction catalogue. The seller enters the item details, which are sent to the system for processing. The system verifies the seller's identity and validates the item information. If the data is valid, the item is saved in the database and added to the catalogue. If validation fails, the system returns an error message and the item is not created.

5 Architecture

5.1 Decomposition Diagram



5.2 Services Table

Services			
Service Name	**Description**	**Exposed Interface Name**	**Interface Description**
M1	User login and registration	M1:I1 M1:I2	M1:I1 Handles user sign in M1:I2 Handles user registration
M2	Manages payment authentication and receipt handling	M2:I3 M2:I4	M2:I3 Handles processing user payment information

			M2:I4 Handles generating receipt for user purchase
M3	Handles auction browsing, search and filtering functionality	M3:I5	M3:I5 Manages searching, filtering and browsing of auction listings
M4	Handles placing bids on auction items	M4:I6	M4:I6 Handles bidding operations
M5	Handles creation of auction listings	M5:I7	M5:I7 Manages the creation of auction listings

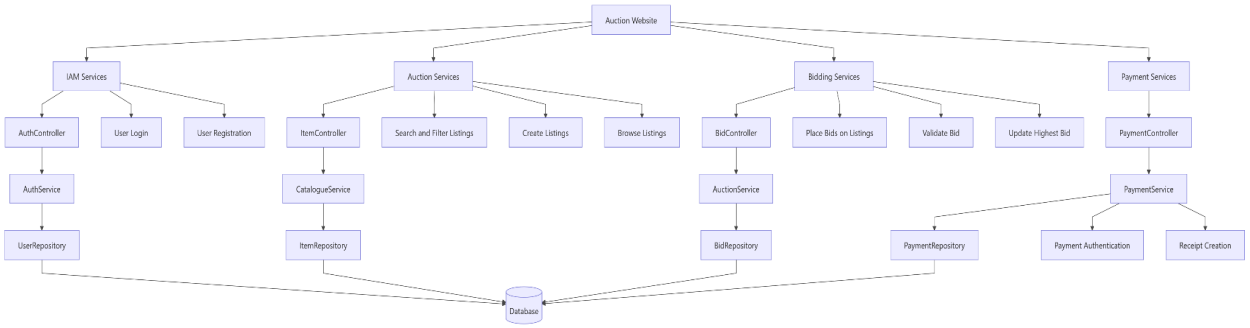
5.3 Interfaces Table

Interfaces		
Interface Name	**Operations**	**Operation Descriptions**
M1:I1	<boolean> Op1(credentials) used by M1	M1:I1:Op1: handles signing into the website with user credentials
M1:I2	<Account> Op2(String userEmail, String userPassword)	M1:I2:Op2: registration interface that handles account creation
M2:I3	<boolean> Op3 (enterPaymentDetails cardInfo) <boolean> Op3 (validatePaymentDetails cardInfo)	M2:I3:Op3: enterPaymentDetails handles submission of payment information M2:I3:Op3: validatePaymentDetails handles processing and authenticating payment
M2:I4	<String> Op4 (String genReceipt) <String> Op4 (String displayReceipt)	M2:I4:Op4: genReceipt handles the creation of the receipt for purchase M2:I4:Op4: displayReceipt displays a viewable and savable copy of the order receipt

M3:I5	<String> Op5 (String searchItems) <String> Op5 (String itemList)	M3:I5:Op5:searchItems handles querying the website database for auction items M3:I5:Op5: itemList shows results from search query if there are any
M4:I6	<boolean> Op6 (int placeBid)	M4:I6:Op6:placeBid handles bidding on auction listings
M5:I7	<listing> Op7 (String listingName, String listingInfo, int startingBid)	M5:I7:Op7: handles the creation of an auction listing

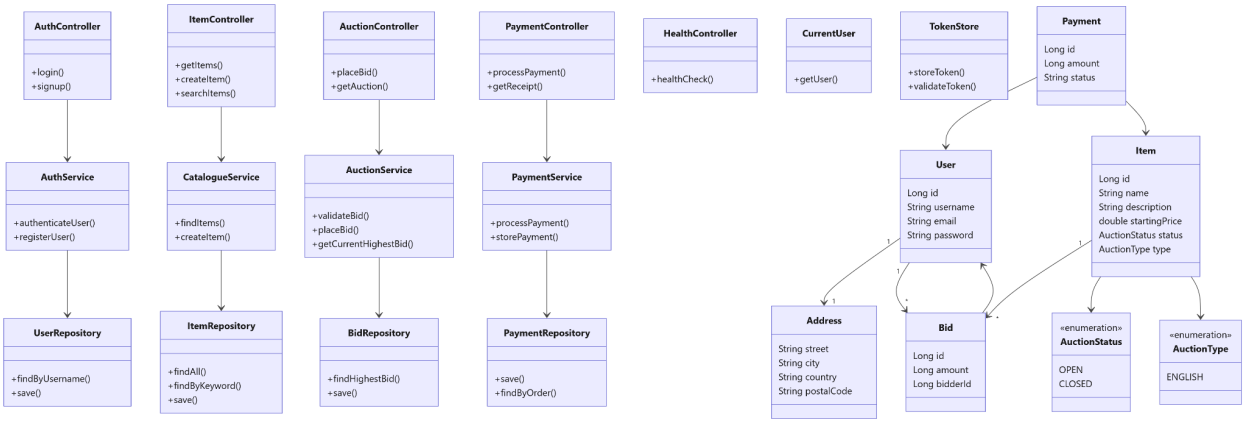
5 (Appended) Architecture

5.1 (Appended) Decomposition Diagram



For a better image of the system architecture diagram click here.

Class Diagram:



For a better image of the class diagram click here.

5.2 (Appended) Services Table

Service Name	Description	Exposed Interface	Interface Description
Authentication Service	Handles user authentication and account creation including login and registration.	M1:I1 – Login Interface M1:I2 – Registration Interface	M1:I1 Handles user sign in M1:I2 Handles user registration
Catalogue Service	Handles browsing, searching, and viewing auction listings as well as creating new listings.	M3:I5 – Listing Browsing Interface	M3:I5 Manages searching, filtering and browsing of auction listings
Auction Service	Manages auction bidding logic and auction state information.	M4:I6 – Bidding Interface	M4:I6 Handles bidding operations
Payment Service	Handles payment processing for auction winners. Generates and displays purchase receipts after payment is completed.	M2:I3 – Payment Interface M2:I4 – Receipt Interface	M2:I3 Handles processing user payment information M2:I4 Handles generating receipt for user purchase

5.3 (Appended) Interfaces Table

Interfaces		
Interface Name	**Operations**	**Operation Descriptions**
Login Interface	showLoginForm() enterCredentials(LoginRequest req) login(LoginRequest req) findByUsername(String username) validatePassword(String password)	Displays the login page to the user. Allows the user to enter login credentials. Sends the login request to the authentication controller.

	LoginResponse displayHomePage() displayError(String message)	Retrieves the user record from the database. Validates the user password. Contains the result of the login attempt. Redirects the user to the system homepage after successful login. Displays an error message if authentication fails.
M1:I2	clickSignUp() showSignUpForm() enterUserDetails(SignupRequest req) signup(SignupRequest req) validateSignup(SignupRequest req) save(User user) SignupResponse displayError(String message) showLoginForm()	User selects the option to register a new account. Displays the sign-up form. User enters registration information. Sends registration request to the authentication controller. Validates the entered registration information. Stores the new user in the database. Contains the result of the registration process. Displays error if registration fails. Redirects user to login page after registration.
M2:I3	enterPaymentDetails(PaymentRequest req) pay(Long itemId, PaymentRequest req) validatePayment(PaymentRequest req)	User enters payment information. Sends payment request to the payment controller.

	save(Payment payment) PaymentResponse displayError(String message)	Verifies payment information. Stores payment record in the database. Returns result of payment transaction. Displays payment error message
M2:I4	viewReceipt(Long receiptId) receipt(Long receiptId) findById(Long receiptId) ReceiptResponse displayReceipt()	User requests receipt for a completed purchase. Retrieves receipt information from payment service. Retrieves payment record associated with the receipt. Contains receipt information. Displays receipt to the user.
M3:I5	enterSearch(String keyword) search(String keyword) findItems(String keyword) queryItems(String keyword) List<ItemSummaryResponse> displayResults(List<ItemSummaryRespon se>) displayNoResults() get(Long itemId) create(CreateItemRequest req) save(Item item)	User enters a search keyword. Sends search request to the item controller. Catalogue service processes the search query. Retrieves matching items from the database. Returns list of items matching the search criteria. Displays matching auction listings. Displays message if no items are found. Retrieves detailed information for a selected item.

		Allows sellers to upload a new auction item
M4:I6	enterBid(BidRequest req) bid(Long itemId, BidRequest req) placeBid(Long itemId, String username, BidRequest req) findTopByItemIdOrderByAmountDesc(Long itemId) validateBid(BidRequest req) save(Bid bid) AuctionStateResponse state(Long itemId) displayUpdatedBid() displayError(String message)	User enters a bid amount. Sends bid request to the auction controller. Processes bid logic in auction service. Retrieves the highest bid for the item. Checks whether bid is valid and higher than current bid. Stores the bid in the database. Returns updated auction state. Retrieves auction status. Displays updated highest bid. Displays bid rejection message.

6 Activities Plan

6.1 Product Backlog for Auction System

#	Requirements/Feature	Priority	Status
1	Login and Registration Page	High	In Progress
2	Profile Management page	High	In Progress
3	Create Listing page	High	In Progress
4	View Listings page	Medium	In Progress

5	Browsing/Searching Listings	Medium	In Progress
6	Placing Bids on Listings	High	In Progress
7	Payment page	High	In Progress
8	Notifications	Medium	In Progress
9	Watchlist Page	Low	In Progress
10	Shipping Page	High	In Progress
11	Auction History	Low	In Progress

6.1 (Appended) Product Backlog for Auction System

#	Requirements/Feature	Priority	Status
1	Login and Registration Page	High	Completed
2	Profile Management page	High	Completed
3	Create Listing page	High	Completed
4	View Listings page	Medium	Completed
5	Browsing/Searching Listings	Medium	Completed
6	Placing Bids on Listings	High	Completed
7	Payment page	High	Completed
8	Notifications	Medium	In Progress
9	Watchlist Page	Low	In Progress
10	Shipping Message	High	Completed
11	Auction History	Low	In Progress
12	Auction End Behaviour	High	Completed
13	Receipt Message	Medium	Completed

6.2 Deliverable 1 Sprint Backlog

#	Section of Software Design Document	Priority	Status
1	Title Page	High	Completed
2	Introduction	High	Completed
3	Major Design Decisions	High	Completed
4	Sequence Diagrams	High	Completed
5	Activity Diagrams	High	Completed
6	Architecture	High	Completed
7	Activities Plan	High	Completed
8	Group Meeting Logs	High	Completed
9	Test Driven Developments	High	Completed
10	AI Acknowledgement	High	Completed

6.3 Deliverable 2 Sprint Backlog

Task	Priority	Status	Feature
UC 1 Implementation	High	In Progress	1-2
UC 2 Implementation	High	In Progress	3-5
UC 3 Implementation	High	In Progress	6
UC 4 Implementation	High	In Progress	7
UC 5 Implementation	High	In Progress	10
UC 6 Implementation	High	In Progress	8,9,11

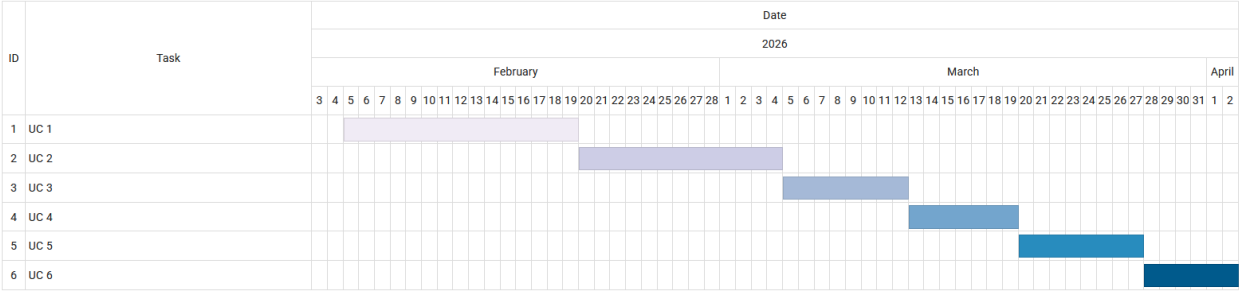
6.3 (Appended) Deliverable 2 Sprint Backlog

Task	Priority	Status	Feature
UC 1 Implementation	High	Completed	1
UC 2 Implementation	High	Completed	5
UC 3 Implementation	High	Completed	6
UC 4 Implementation	High	Completed	12
UC 5 Implementation	High	Completed	7
UC 6 Implementation	High	Completed	10, 13
UC 7 Implementation	High	Completed	3

6.4 Deliverable 3 Sprint Backlog

Task	Priority	Status	Feature and Requirement
Full System Functional and Tested	High	In Progress	1-11
Additional Features Implemented	High	In Progress	9
GUI Completed	High	In Progress	1-13

6.5 Gantt Diagram



7 Group Meeting Logs

Meeting 1 – Project Kickoff and Task Allocation
The first group meeting was held on Tuesday, January 27, where we reviewed the course project and Deliverable 1 requirements in detail. During this meeting, we discussed the scope of the deliverable, including the required UML diagrams, architecture specification, activity plan, and test cases. The group aligned on the overall approach and identified the key activities needed to complete Deliverable 1. Tasks were then assigned to each group member based on the required sections of the deliverable. Following this meeting, the team continued to collaborate asynchronously through the Discord group channel by sharing progress updates, providing feedback, and coordinating work.

Meeting 2 – Progress Review and Internal Deadline Check
The second meeting was held on Wednesday, February 4, which served as the group's internal soft deadline for Deliverable 1. During this meeting, members presented their completed or in-progress sections, and the group reviewed the work collectively for completeness and consistency. The team also cross-checked the deliverable against the grading criteria posted on eClass to ensure alignment with the evaluation requirements. Feedback was shared, and final refinements were discussed to prepare the deliverable for submission.

7 (Appended) Group Meeting Logs

Meeting 3 – Deliverable 2 Discussion and Task Assignment
The third meeting was held on Friday, February 20, where the group discussed the second part of the project and what was required for Deliverable 2. During this meeting the group organized what needed to be done, how it would be done and who would be doing it. The group discussed and aligned the overall approach we would be taking to complete the project as well as setting up an internal deadline for all tasks to be completed by. Afterwards, group members were assigned tasks to complete. Following this, the group collaborated asynchronously through the team discord server.

Meeting 4 – Deliverable 2 Progress Report and Internal Deadline
The fourth meeting was held on Monday, March 2, where the group met up in accordance with the internal deadline set during the first meeting. Here we checked in, presented the work done and cross checked each other's work. The team then made final adjustments and alterations based on feedback given to each other and prepared for submission.

8 Test Driven Development (TDD)

8.1 User Authentication

Test ID	TC-01
Category	User Authentication
Requirements Coverage	UC1-Login
Initial Condition	System running, DB contains user Alice
Procedure	1. User selects login 2. Enter username "Alice" 3. Enter correct password 4. Submit
Expected Outcome	Login successful, main UI appears
Notes	Username are password alphanumeric only

Test ID	TC-02
Category	User Authentication
Requirements Coverage	UC1-Login
Initial Condition	System running, DB contains user Bob
Procedure	1. Enter username "Bob" 2. Enter wrong password 3. Submit
Expected Outcome	Error message shown, login denied
Notes	Test incorrect credentials

Test ID	TC-03
Category	User Authentication
Requirements Coverage	UC1-Login

Initial Condition	System running, DB contains multiple users
Procedure	1. Leave username blank 2. Enter password 3. Submit
Expected Outcome	Error message: "Username required"
Notes	Blank field validation

Test ID	TC-04
Category	User Authentication
Requirements Coverage	UC1-Login
Initial Condition	System running
Procedure	1. Enter special characters in username 2. Enter password 3. Submit
Expected Outcome	Error message: "Invalid characters"
Notes	Security check

8.2 User Registration

Test ID	TC-05
Category	User Registration
Requirements Coverage	UC2-Register
Initial Condition	System running
Procedure	1. Enter new username "Carol" 2. Enter password "pass123" 3. Submit

Expected Outcome	Account created, user can login
Notes	Username must be unique

Test ID	TC-06
Category	User Registration
Requirements Coverage	UC2-Register
Initial Condition	System running
Procedure	1. Enter existing username "Alice" 2. Enter password 3. Submit
Expected Outcome	Error message: "Username exists"
Notes	Unique username validation

Test ID	TC-07
Category	User Registration
Requirements Coverage	UC2-Register
Initial Condition	System running
Procedure	1. Enter password <6 characters 2. Submit
Expected Outcome	Error: "Password too short"
Notes	Password length validation

Test ID	TC-08
Category	User Registration
Requirements Coverage	UC2-Register
Initial Condition	System running

Procedure	1. Enter password with invalid characters 2. Submit
Expected Outcome	Error message
Notes	Test allowed characters only

8.3 Data Entry Form

Test ID	TC-9
Category	Data Entry Form
Requirements Coverage	UC3-Form
Initial Condition	System running, user logged in
Procedure	1. Open data form 2. Enter valid data 3. Submit
Expected Outcome	Data saved to DB
Notes	Field type validation

Test ID	TC-10
Category	Data Entry Form
Requirements Coverage	UC3-Form
Initial Condition	System running, user logged in
Procedure	1. Leave required field blank 2. Submit
Expected Outcome	Error message: "Field required"
Notes	Required field check

Test ID	TC-11

Category	Data Entry Form
Requirements Coverage	UC3-Form
Initial Condition	System running, user logged in
Procedure	1. Enter invalid data type (e.g., letters in numeric field) 2. Submit
Expected Outcome	Error message: "Invalid input"
Notes	Data type validation

Test ID	TC-12
Category	Data Entry Form
Requirements Coverage	UC3-Form
Initial Condition	System running, user logged in
Procedure	Submit form multiple times quickly
Expected Outcome	System prevents duplicate entries
Notes	Prevent accidental duplicates

8.4 Reporting

Test ID	TC-13
Category	Reporting
Requirements Coverage	UC4-Reporting
Initial Condition	System running, data exists
Procedure	1. Select report type 2. Generate report
Expected Outcome	Report generated with correct data
Notes	Test report accuracy

Test ID	TC-14
Category	Reporting
Requirements Coverage	UC4-Reporting
Initial Condition	System running, no data available
Procedure	Generate report
Expected Outcome	Message: "No data available"
Notes	Empty data edge case

Test ID	TC-15
Category	Reporting
Requirements Coverage	UC4-Reporting
Initial Condition	System running, data exists
Procedure	1. Generate report 2. Export to CSV
Expected Outcome	CSV file created with correct data
Notes	Test export functionality

Test ID	TC-16
Category	Reporting
Requirements Coverage	UC4-Reporting
Initial Condition	System running, multiple reports exist
Procedure	Generate multiple reports sequentially
Expected Outcome	Reports correctly generated without conflicts
Notes	Test system stability

8.5 User Profile

Test ID	TC-17
Category	User Profile
Requirements Coverage	UC5-User Profile
Initial Condition	System running, user logged in
Procedure	User navigates to profile page
Expected Outcome	User profile information is displayed correctly
Notes	Profile data fetched from DB

Test ID	TC-18
Category	User Profile
Requirements Coverage	UC5-User Profile
Initial Condition	System running, user logged in
Procedure	1. User edits profile fields 2. Clicks save
Expected Outcome	Profile updated successfully
Notes	Validate editable fields

Test ID	TC-19
Category	User Profile
Requirements Coverage	UC5-User Profile
Initial Condition	System running, user logged in
Procedure	1. Enter invalid email format 2. Click save
Expected Outcome	Email format validation

Notes	Validate editable fields

Test ID	TC-20
Category	User Profile
Requirements Coverage	UC5-User Profile
Initial Condition	System running, user logged in
Procedure	1. Enter old password 2. Enter new valid password 3. Submit
Expected Outcome	Password updated successfully
Notes	Password policy enforced

8 (Appended) Test Driven Development (TDD)

8.1 (Appended) User, Sign Up, Login and Authentication (UC1)

Test ID	TC-01: Successful Login
Category	User Authentication
Requirements Coverage	UC1-Sign Up and Sign In
Initial Condition	System running, DB contains user “yazan” with valid password
Procedure	1. User selects login 2. Enter username "yazan" 3. Enter correct password 4. Submit
Expected Outcome	Login successful, authentication token issued, user session starts
Notes	Password must match stored hashed password

Test ID	TC-02: Login With Non-Existing User
Category	User Authentication
Requirements Coverage	UC1-Sign Up and Sign In
Initial Condition	System running, username "yazan" does not exist in DB
Procedure	1. User selects login 2. Enter username "yazan" 3. Enter password 4. Submit
Expected Outcome	Login fails, error returned indicating user does not exist
Notes	System must not issue authentication token

Test ID	TC-03:Login With Incorrect Password
Category	User Authentication
Requirements Coverage	UC1-Sign Up and Sign In
Initial Condition	System running, DB contains user yazan
Procedure	1. User selects login 2. Enter username "yazan" 3. Enter incorrect password 4. Submit
Expected Outcome	Login fails, error returned indicating invalid credentials
Notes	Authentication token must not be issued

Test ID	TC-04: Signup With Duplicate Username
Category	User Authentication
Requirements Coverage	UC1-Sign Up and Sign In
Initial Condition	System running, username "yazan" already exists in DB

Procedure	1. User selects signup 2. Enter username "yazan" 3. Enter valid password 4. Enter all required personal and shipping information 5. Submit
Expected Outcome	Signup fails, error returned indicating username already exists
Notes	No new user record should be saved

Test ID	TC-05: Successful Signup
Category	User Authentication
Requirements Coverage	UC1-Sign Up and Sign In
Initial Condition	System running, username "yazan" does not already exist in DB
Procedure	1. User selects signup 2. Enter username "yazan" 3. Enter password "StrongPass123!" 4. Enter first name "Yazan" 5. Enter last name "A" 6. Enter shipping address details 7. Submit
Expected Outcome	Account created successfully, user record saved, unique user ID assigned
Notes	Password should be stored as a hash, not plain text

Test ID	TC-06: Signup Missing Shipping Address
Category	User Authentication
Requirements Coverage	UC1-Sign Up and Sign In
Initial Condition	System running, username does not already exist in DB
Procedure	1. User selects signup 2. Enter valid username 3. Enter valid password 4. Enter first and last name 5. Leave shipping address empty 6. Submit
Expected Outcome	Signup fails, error returned because shipping address is required
Notes	User record must not be saved without required address information

8.2 (Appended) Placing and Validating Bids (UC3)

Test ID	TC-07: View Auction State Successfully

Category	Placing and Validating Bids
Requirements Coverage	UC3-Placing and Validating Bids
Initial Condition	System running, item exists, auction is active
Procedure	1. User opens auction item 2. System retrieves current auction state
Expected Outcome	Auction state is returned with item ID, status, current bid, highest bidder, and end time
Notes	Supports bid validation by showing current auction info

Test ID	TC-08: View Auction State for Non-Existing Item
Category	Placing and Validating Bids
Requirements Coverage	UC3-Placing and Validating Bids
Initial Condition	System running, requested item does not exist
Procedure	1. User opens auction item 2. System attempts to retrieve auction state
Expected Outcome	Request fails with item not found error
Notes	Prevents bidding on invalid item IDs

Test ID	TC-09: Place Bid Successfully
Category	Placing and Validating Bids
Requirements Coverage	UC3-Placing and Validating Bids
Initial Condition	System running, item exists, auction is active, new bid is greater than current bid
Procedure	1. User selects an active auction 2. Enter bid amount higher than current bid 3. Submit bid
Expected Outcome	Bid is accepted, current bid updates, highest bidder updates, bid record is saved

Notes	Auction remains ACTIVE after successful bid

Test ID	TC-10: Place Bid on Non-Existing Item
Category	Placing and Validating Bids
Requirements Coverage	UC3-Placing and Validating Bids
Initial Condition	System running, item does not exist
Procedure	1. User enters bid for invalid item ID 2. Submit bid
Expected Outcome	Bid is rejected with item not found error
Notes	No bid record should be saved

Test ID	TC-11: Place Bid on Ended Auction
Category	Placing and Validating Bids
Requirements Coverage	UC3-Placing and Validating Bids
Initial Condition	System running, item exists, auction status is ENDED
Procedure	1. User enters bid on ended auction 2. Submit bid
Expected Outcome	Bid is rejected
Notes	No bid record should be saved

Test ID	TC-12: Place Bid on Removed Auction With No Bids
Category	Placing and Validating Bids
Requirements Coverage	UC3-Placing and Validating Bids
Initial Condition	System running, item exists, auction status is REMOVED_NO_BIDS

Procedure	1. User enters bid on removed auction 2. Submit bid
Expected Outcome	Bid is rejected
Notes	No bid record should be saved

Test ID	TC-13: Place Bid Equal to Current Bid
Category	Placing and Validating Bids
Requirements Coverage	UC3-Placing and Validating Bids
Initial Condition	System running, item exists, auction active, current bid already set
Procedure	1. User enters bid equal to current bid 2. Submit bid
Expected Outcome	Bid is rejected
Notes	Bid must be strictly greater than current bid

Test ID	TC-14: Place Bid Lower Than Current Bid
Category	Placing and Validating Bids
Requirements Coverage	UC3-Placing and Validating Bids
Initial Condition	System running, item exists, auction active, current bid already set
Procedure	1. User enters bid lower than current bid 2. Submit bid
Expected Outcome	Bid is rejected
Notes	No bid record or item update should occur

Test ID	TC-15: Place Bid on Expired Auction That Had Bids
Category	Placing and Validating Bids
Requirements Coverage	UC3-Placing and Validating Bids

Initial Condition	System running, item exists, auction end time has passed, item already has bids
Procedure	1. User enters bid after auction expiry 2. Submit bid
Expected Outcome	Bid is rejected, auction status updates to ENDED
Notes	No new bid should be saved

Test ID	TC-16: Place Bid on Expired Auction With No Bids
Category	Placing and Validating Bids
Requirements Coverage	UC3-Placing and Validating Bids
Initial Condition	System running, item exists, auction end time has passed, item has no bids
Procedure	1. User enters bid after auction expiry 2. Submit bid
Expected Outcome	Bid is rejected, auction status updates to REMOVED_NO_BIDS
Notes	No new bid should be saved

8.3 (Appended) Auction End Behaviour (UC4)

Test ID	TC-17: Expired Auction With No Bids Is Removed
Category	Auction Lifecycle
Requirements Coverage	UC4-Auction End Behaviour
Initial Condition	System running, auction is active, end time has passed, no bids were placed
Procedure	System refreshes auction status after expiry
Expected Outcome	Auction status changes to REMOVED_NO_BIDS
Notes	Updated status should be saved

Test ID	TC-18: Expired Auction With Bids Is Ended
Category	Auction Lifecycle
Requirements Coverage	UC4-Auction End Behaviour
Initial Condition	System running, auction is active, end time has passed, at least one bid exists
Procedure	System refreshes auction status after expiry
Expected Outcome	Auction status changes to ENDED
Notes	Updated status should be saved

Test ID	TC-19: Active Future Auction Remains Active
Category	Auction Lifecycle
Requirements Coverage	UC4-Auction End Behaviour
Initial Condition	System running, auction is active, end time has not yet passed
Procedure	System refreshes auction status before expiry
Expected Outcome	Auction remains ACTIVE
Notes	No save/update should occur

8.4 (Appended) Payment (UC5)

Test ID	TC-20: Successful Payment Without Expedited Shipping
Category	Payment
Requirements Coverage	UC5-Payment
Initial Condition	System running, item exists, auction status is ENDED, user is the winning bidder
Procedure	1. User proceeds to payment for won item 2. Enter valid card details 3. Select normal shipping 4. Submit payment

Expected Outcome	Payment succeeds, receipt is generated, item price and normal shipping cost are included in total
Notes	Total paid = item price + normal shipping cost

Test ID	TC-21: Successful Payment With Expedited Shipping
Category	Payment
Requirements Coverage	UC5-Payment
Initial Condition	System running, item exists, auction status is ENDED, user is the winning bidder, expedited shipping is available
Procedure	1. User proceeds to payment for won item 2. Enter valid card details 3. Select expedited shipping 4. Submit payment
Expected Outcome	Payment succeeds, receipt is generated, total includes item price plus both regular and expedited shipping cost
Notes	Shipping cost increases when expedited shipping is selected

Test ID	TC-22: Payment for Non-Existing Item
Category	Payment
Requirements Coverage	UC5-Payment
Initial Condition	System running, requested item ID does not exist
Procedure	1. User attempts payment for invalid item ID 2. Submit payment
Expected Outcome	Payment fails with item not found error
Notes	No payment record should be saved

Test ID	TC-23: Payment for Auction Removed With No Bids
Category	Payment
Requirements Coverage	UC5-Payment
Initial Condition	System running, item exists, auction status is REMOVED_NO_BIDS

Procedure	1. User attempts payment for removed auction item 2. Submit payment
Expected Outcome	Payment fails
Notes	Removed auctions cannot be paid for

Test ID	TC-24: Payment Before Auction Ends
Category	Payment
Requirements Coverage	UC5-Payment
Initial Condition	System running, item exists, auction status is ACTIVE
Procedure	1. User attempts payment before auction is finished 2. Submit payment
Expected Outcome	Payment fails
Notes	Payment is only allowed after auction ends

Test ID	TC-25: Non-Winning Bidder Attempts Payment
Category	Payment
Requirements Coverage	UC5-Payment
Initial Condition	System running, item exists, auction status is ENDED, logged-in user is not the highest bidder
Procedure	1. Non-winning user attempts to pay for item 2. Submit payment
Expected Outcome	Payment fails
Notes	Only the winning bidder may complete payment

Test ID	TC-26: Payment Rejected When No Winner Exists
Category	Payment
Requirements Coverage	UC5-Payment
Initial	System running, auction status is ENDED, highest bidder is null

Condition	
Procedure	1. User attempts payment for ended auction with no valid winner 2. Submit payment
Expected Outcome	Payment fails
Notes	System must reject payment if winner information is missing

Test ID	TC-27: Payment Refreshes Auction Status Before Processing
Category	Payment
Requirements Coverage	UC5-Payment
Initial Condition	System running, item exists, payment request is valid
Procedure	1. User submits payment for won item 2. System refreshes auction status before finalizing payment
Expected Outcome	Auction status is refreshed and payment proceeds only if status remains valid
Notes	Ensures payment logic uses the latest auction state

8.5 (Appended) Receipt and Shipping (UC6)

Test ID	TC-28: Receipt Retrieved Successfully
Category	Receipt and Shipping
Requirements Coverage	UC6-Receipt and Shipping
Initial Condition	System running, payment record exists with valid receipt ID
Procedure	User requests receipt using valid receipt ID
Expected Outcome	Receipt is returned with receipt ID, total paid, and shipping information
Notes	Shipping message includes estimated delivery time

Test ID	TC-29: Receipt Request for Invalid Receipt ID
Category	Receipt and Shipping

Requirements Coverage	UC6-Receipt and Shipping
Initial Condition	System running, requested receipt ID does not exist
Procedure	User requests receipt using invalid receipt ID
Expected Outcome	Request fails with receipt not found error
Notes	No receipt data should be returned

9 (Appended) Test plan Summary

The tests are split between running using Postman Collections to test the End-To-End Flow (Happy Path in the README) and to test the Robustness (Negative Tests in the README) test cases in the tests file. The Happy Path covers UC1-UC7 which encompasses Sign Up and In, Browse and Searching the Catalogue, whether bids are valid, Auction End Behaviour, Payment, Receipt and Shipping Messages, and Creating a Catalogue Item. The Negative Path tests cover errors that can be expected when using the system as normal such as duplicate usernames while signing up, invalid login details, invalid bid amounts, bidding after an auction has ended, a non-winning attempting to pay and missing payment fields. The tests in the test case file cover the same things as previously stated just in the form of test cases as opposed to using Postman.

The following changes were made in the TDD with respect to the JUnit test cases:

- 8.2 tests are added to 8.1 to test for UC1 (Sign Up & Log In), 8.2 is now for UC3 (Placing and Validating Bids)
- 8.3 is now representing UC4 (Auction End Behaviour) instead of Data Entry
- 8.4 is now representing UC5 (Payment) instead of Reporting
- 8.5 is now representing UC6 (Receipt and Shipping) instead of User Profile

10 AI Acknowledgement

AI tools were used to support productivity during this project, primarily for brainstorming, documentation, and refining written content. All AI-assisted artifacts were reviewed, understood, critiqued, and modified by the team. The final design decisions, implementations, and maintenance of the project remain the responsibility of the authors.

11 (Appended) Deviations from Original Design

The implemented system preserved the overall modular and service-oriented direction proposed in Deliverable 1, but several refinements were made during implementation to better align the design with the actual Spring Boot backend.

First, the abstract service naming used in Deliverable 1, such as M1, M2, and M3, was replaced with concrete implementation-oriented components including AuthController, AuthService, ItemController, CatalogueService, AuctionService, PaymentService, and their associated repositories. This change improved clarity and created direct traceability between the design document and the implemented codebase.

Second, the implementation adopted a layered Spring Boot architecture consisting of Controllers, Services, Repositories, DTOs, and Entities. While the original design described a modular service-based system, the implemented architecture refined this into a concrete layered structure that better supports maintainability, separation of concerns, and testing.

Third, token-based stateless authentication was introduced in the implementation to support protected API access. This was a refinement of the original authentication design and was chosen to better support REST principles and future microservice deployment.

Finally, the architecture tables, decomposition diagrams, and sequence diagrams were updated to reflect the actual implementation details and naming conventions used in the backend. These changes do not represent a change in the core system goals, but rather an implementation-level refinement of the original design.

12 (Appended) Amendments to the Original Document

- Changed the decomposition diagram in 5.1 of Architecture to reflect the current system Architecture including service diagrams
- Fixed the Services Table (5.2) and the Interfaces Table (5.3) to better reflect the names of variables and design changes made in the implementation
- Appended the group meeting log (section 7) with what was discussed in the meetings
- Added and updated the test plan summary (section 9) to reflect what the Postman and test cases do and what they test in the implementation
- Added the “Deviations from Original Design” section (section 11) that discusses and lists any changes made to the implementation architecture
- Appended a section going over the design patterns used and considered (section 2 (Appended) Key Design Choices)
- Appended 6.1 to better reflect the systems that needed to be implemented
- Appended 6.3 to better reflect UC1-UC7 implementation